

Manager	Mean, $\bar{x}$	Standard Deviation, s	t -statistic	p -value
One	3.0	7.4	2.86	0.006
Two	-0.1	6.9	-0.10	0.918
Three	2.8	7.5	2.62	0.012
Four	0.5	6.7	0.53	0.601
Five	0.3	6.8	0.31	0.756

TABLE 13.3. The first two columns correspond to the sample mean and sample standard deviation of the percentage excess return, over $n = 50$ months, for the first five managers in the **Fund** dataset. The last two columns provide the t -statistic ($\sqrt{n} \cdot \bar{X}/S$) and associated p -value for testing $H_{0j} : \mu_j = 0$, the null hypothesis that the (population) mean return for the j th hedge fund manager equals zero.

13.3.2 Approaches to Control the Family-Wise Error Rate

In this section, we briefly survey some approaches to control the FWER. We will illustrate these approaches on the **Fund** dataset, which records the monthly percentage excess returns for 2,000 fund managers over $n = 50$ months.¹³ Table 13.3 provides relevant summary statistics for the first five managers.

We first present the Bonferroni method and Holm's step-down procedure, which are very general-purpose approaches for controlling the FWER that can be applied whenever m p -values have been computed, regardless of the form of the null hypotheses, the choice of test statistics, or the (in)dependence of the p -values. We then briefly discuss Tukey's method and Scheffé's method in order to illustrate the fact that, in certain situations, more specialized approaches for controlling the FWER may be preferable.

The Bonferroni Method

As in the previous section, suppose we wish to test $H_{01}, \dots, H_{0m}$. Let A_j denote the event that we make a Type I error for the j th null hypothesis, for $j = 1, \dots, m$. Then

$$\begin{aligned}
 \text{FWER} &= \Pr(\text{falsely reject at least one null hypothesis}) \\
 &= \Pr(\cup_{j=1}^m A_j) \\
 &\leq \sum_{j=1}^m \Pr(A_j).
 \end{aligned} \tag{13.6}$$

In (13.6), the inequality results from the fact that for any two events A and B , $\Pr(A \cup B) \leq \Pr(A) + \Pr(B)$, regardless of whether A and B are independent. The *Bonferroni method*, or *Bonferroni correction*, sets the threshold for rejecting each hypothesis test to α/m , so that $\Pr(A_j) \leq \alpha/m$. Equation 13.6 implies that

$$\text{FWER}(\alpha/m) \leq m \times \frac{\alpha}{m} = \alpha,$$

¹³Excess returns correspond to the additional return the fund manager achieves beyond the market's overall return. So if the market increases by 5% during a given period and the fund manager achieves a 7% return, their *excess return* would be $7\% - 5\% = 2\%$.

so this procedure controls the FWER at level α . For instance, in order to control the FWER at level 0.1 while testing $m = 100$ null hypotheses, the Bonferroni procedure requires us to control the Type I error for each null hypothesis at level $0.1/100 = 0.001$, i.e. to reject all null hypotheses for which the p -value is below 0.001.

We now consider the **Fund** dataset in Table 13.3. If we control the Type I error at level $\alpha = 0.05$ for each fund manager separately, then we will conclude that the first and third managers have significantly non-zero excess returns; in other words, we will reject $H_{01} : \mu_1 = 0$ and $H_{03} : \mu_3 = 0$. However, as discussed in previous sections, this procedure does not account for the fact that we have tested multiple hypotheses, and therefore it will lead to a FWER greater than 0.05. If we instead wish to control the FWER at level 0.05, then, using a Bonferroni correction, we must control the Type I error for each individual manager at level $\alpha/m = 0.05/5 = 0.01$. Consequently, we will reject the null hypothesis only for the first manager, since the p -values for all other managers exceed 0.01. The Bonferroni correction gives us peace of mind that we have not falsely rejected too many null hypotheses, but for a price: we reject few null hypotheses, and thus will typically make quite a few Type II errors.

The Bonferroni correction is by far the best-known and most commonly-used multiplicity correction in all of statistics. Its ubiquity is due in large part to the fact that it is very easy to understand and simple to implement, and also from the fact that it successfully controls Type I error regardless of whether the m hypothesis tests are independent. However, as we will see, it is typically neither the most powerful nor the best approach for multiple testing correction. In particular, the Bonferroni correction can be quite conservative, in the sense that the true FWER is often quite a bit lower than the nominal (or target) FWER; this results from the inequality in (13.6). By contrast, a less conservative procedure might allow us to control the FWER while rejecting more null hypotheses, and therefore making fewer Type II errors.

Holm's Step-Down Procedure

Holm's method, also known as Holm's step-down procedure or the Holm–Bonferroni method, is an alternative to the Bonferroni procedure. Holm's method controls the FWER, but it is less conservative than Bonferroni, in the sense that it will reject more null hypotheses, typically resulting in fewer Type II errors and hence greater power. The procedure is summarized in Algorithm 13.1. The proof that this method controls the FWER is similar to, but slightly more complicated than, the argument in (13.6) that the Bonferroni method controls the FWER. It is worth noting that in Holm's procedure, the threshold that we use to reject each null hypothesis — $p_{(L)}$ in Step 5 — actually depends on the values of *all* m of the p -values. (See the definition of L in (13.7).) This is in contrast to the Bonferroni procedure, in which to control the FWER at level α , we reject any null hypotheses for which the p -value is below α/m , regardless of the other p -values. Holm's method makes no independence assumptions about the m hypothesis tests, and is uniformly more powerful than the Bonferroni method — it will

Holm's
method

Algorithm 13.1 *Holm's Step-Down Procedure to Control the FWER*

1. Specify α , the level at which to control the FWER.
2. Compute p -values, $p_1, \dots, p_m$, for the m null hypotheses $H_{01}, \dots, H_{0m}$.
3. Order the m p -values so that $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
4. Define

$$L = \min \left\{ j : p_{(j)} > \frac{\alpha}{m+1-j} \right\}. \quad (13.7)$$
5. Reject all null hypotheses H_{0j} for which $p_j < p_{(L)}$.

always reject at least as many null hypotheses as Bonferroni — and so it should always be preferred.

We now consider applying Holm's method to the first five fund managers in the **Fund** dataset in Table 13.3, while controlling the FWER at level 0.05. The ordered p -values are $p_{(1)} = 0.006$, $p_{(2)} = 0.012$, $p_{(3)} = 0.601$, $p_{(4)} = 0.756$ and $p_{(5)} = 0.918$. The Holm procedure rejects the first two null hypotheses, because $p_{(1)} = 0.006 < 0.05/(5+1-1) = 0.01$ and $p_{(2)} = 0.012 < 0.05/(5+1-2) = 0.0125$, but $p_{(3)} = 0.601 > 0.05/(5+1-3) = 0.0167$, which implies that $L = 3$. We note that, in this setting, Holm is more powerful than Bonferroni: the former rejects the null hypotheses for the first and third managers, whereas the latter rejects the null hypothesis only for the first manager.

Figure 13.3 provides an illustration of the Bonferroni and Holm methods on three simulated data sets in a setting involving $m = 10$ hypothesis tests, of which $m_0 = 2$ of the null hypotheses are true. Each panel displays the ten corresponding p -values, ordered from smallest to largest, and plotted on a log scale. The eight red points represent the false null hypotheses, and the two black points represent the true null hypotheses. We wish to control the FWER at level 0.05. The Bonferroni procedure requires us to reject all null hypotheses for which the p -value is below 0.005; this is represented by the black horizontal line. The Holm procedure requires us to reject all null hypotheses that fall below the blue line. The blue line always lies above the black line, so Holm will always reject more tests than Bonferroni; the region between the two lines corresponds to the hypotheses that are only rejected by Holm. In the left-hand panel, both Bonferroni and Holm successfully reject seven of the eight false null hypotheses. In the center panel, Holm successfully rejects all eight of the false null hypotheses, while Bonferroni fails to reject one. In the right-hand panel, Bonferroni only rejects three of the false null hypotheses, while Holm rejects all eight. Neither Bonferroni nor Holm makes any Type I errors in these examples.

Two Special Cases: Tukey's Method and Scheffé's Method

Bonferroni's method and Holm's method can be used in virtually any setting in which we wish to control the FWER for m null hypotheses: they



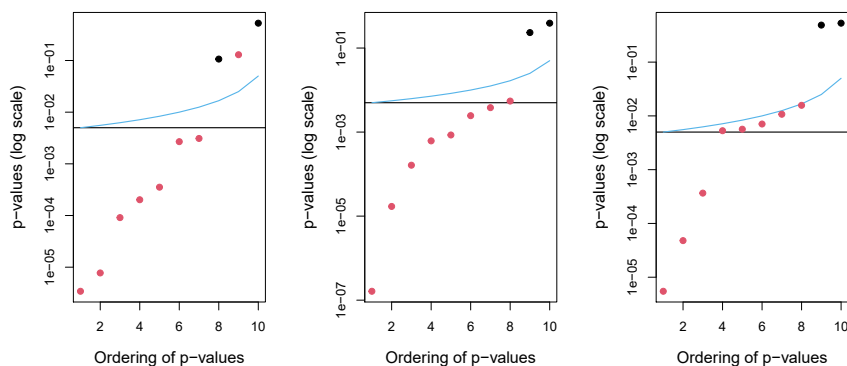


FIGURE 13.3. Each panel displays, for a separate simulation, the sorted p -values for tests of $m = 10$ null hypotheses. The p -values corresponding to the $m_0 = 2$ true null hypotheses are displayed in black, and the rest are in red. When controlling the FWER at level 0.05, the Bonferroni procedure rejects all null hypotheses that fall below the black line, and the Holm procedure rejects all null hypotheses that fall below the blue line. The region between the blue and black lines indicates null hypotheses that are rejected using the Holm procedure but not using the Bonferroni procedure. In the center panel, the Holm procedure rejects one more null hypothesis than the Bonferroni procedure. In the right-hand panel, it rejects five more null hypotheses.

make no assumptions about the nature of the null hypotheses, the type of test statistic used, or the (in)dependence of the p -values. However, in certain very specific settings, we can achieve higher power by controlling the FWER using approaches that are more tailored to the task at hand. Tukey's method and Scheffé's method provide two such examples.

Table 13.3 indicates that for the **Fund** dataset, Managers One and Two have the greatest difference in their sample mean returns. This finding might motivate us to test the null hypothesis $H_0 : \mu_1 = \mu_2$, where μ_j is the (population) mean return for the j th fund manager. A two-sample t -test (13.1) for H_0 yields a p -value of 0.0349, suggesting modest evidence against H_0 . However, this p -value is misleading, since we decided to compare the average returns of Managers One and Two only after having examined the returns for all five managers; this essentially amounts to having performed $m = 5 \times (5 - 1)/2 = 10$ hypothesis tests, and selecting the one with the smallest p -value. This suggests that in order to control the FWER at level 0.05, we should make a Bonferroni correction for $m = 10$ hypothesis tests, and therefore should only reject a null hypothesis for which the p -value is below 0.005. If we do this, then we will be unable to reject the null hypothesis that Managers One and Two have identical performance.

However, in this setting, a Bonferroni correction is actually a bit too stringent, since it fails to consider the fact that the $m = 10$ hypothesis tests are all somewhat related: for instance, Managers Two and Five have similar mean returns, as do Managers Two and Four; this guarantees that the mean returns of Managers Four and Five are similar. Stated another way, the m p -values for the m pairwise comparisons are *not* independent. Therefore, it should be possible to control the FWER in a way that is

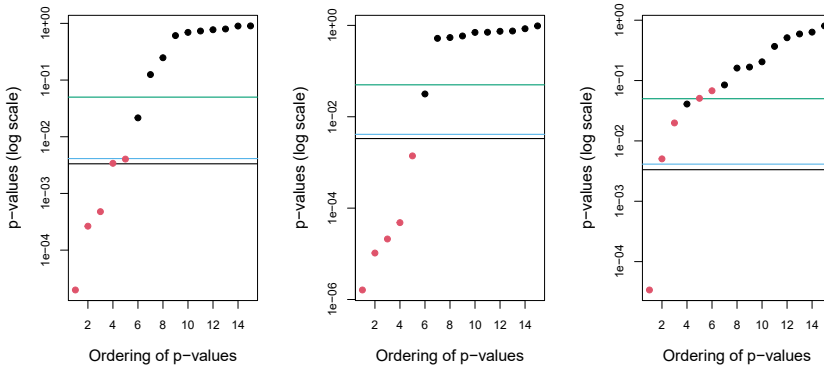


FIGURE 13.4. Each panel displays, for a separate simulation, the sorted p -values for tests of $m = 15$ hypotheses, corresponding to pairwise tests for the equality of $G = 6$ means. The $m_0 = 10$ true null hypotheses are displayed in black, and the rest are in red. When controlling the FWER at level 0.05, the Bonferroni procedure rejects all null hypotheses that fall below the black line, whereas Tukey rejects all those that fall below the blue line. Thus, Tukey's method has slightly higher power than Bonferroni's method. Controlling the Type I error without adjusting for multiple testing involves rejecting all those that fall below the green line.

less conservative. This is exactly the idea behind *Tukey's method*: when performing $m = G(G - 1)/2$ pairwise comparisons of G means, it allows us to control the FWER at level α while rejecting all null hypotheses for which the p -value falls below α_T , for some $\alpha_T > \alpha/m$.

Tukey's
method

Figure 13.4 illustrates Tukey's method on three simulated data sets in a setting with $G = 6$ means, with $\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5 \neq \mu_6$. Therefore, of the $m = G(G - 1)/2 = 15$ null hypotheses of the form $H_0 : \mu_j = \mu_k$, ten are true and five are false. In each panel, the true null hypotheses are displayed in black, and the false ones are in red. The horizontal lines indicate that Tukey's method always results in at least as many rejections as Bonferroni's method. In the left-hand panel, Tukey correctly rejects two more null hypotheses than Bonferroni.

Now, suppose that we once again examine the data in Table 13.3, and notice that Managers One and Three have higher mean returns than Managers Two, Four, and Five. This might motivate us to test the null hypothesis

$$H_0 : \frac{1}{2}(\mu_1 + \mu_3) = \frac{1}{3}(\mu_2 + \mu_4 + \mu_5). \quad (13.8)$$

(Recall that μ_j is the population mean return for the j th hedge fund manager.) It turns out that we could test (13.8) using a variant of the two-sample t -test presented in (13.1), leading to a p -value of 0.004. This suggests strong evidence of a difference between Managers One and Three compared to Managers Two, Four, and Five. However, there is a problem: we decided to test the null hypothesis in (13.8) only after peeking at the data in Table 13.3. In a sense, this means that we have conducted multiple testing. In this setting, using Bonferroni to control the FWER at level α

would require a p -value threshold of α/m , for an extremely large value of m ¹⁴.

Scheffé's method is designed for exactly this setting. It allows us to compute a value α_S such that rejecting the null hypothesis H_0 in (13.8) if the p -value is below α_S will control the Type I error at level α . It turns out that for the **Fund** example, in order to control the Type I error at level $\alpha = 0.05$, we must set $\alpha_S = 0.002$. Therefore, we are unable to reject H_0 in (13.8), despite the apparently very small p -value of 0.004. An important advantage of Scheffé's method is that we can use this same threshold of $\alpha_S = 0.002$ in order to perform a pairwise comparison of any split of the managers into two groups: for instance, we could also test $H_0 : \frac{1}{3}(\mu_1 + \mu_2 + \mu_3) = \frac{1}{2}(\mu_4 + \mu_5)$ and $H_0 : \frac{1}{4}(\mu_1 + \mu_2 + \mu_3 + \mu_4) = \mu_5$ using the same threshold of 0.002, without needing to further adjust for multiple testing.

Scheffé's
method

To summarize, Holm's procedure and Bonferroni's procedure are very general approaches for multiple testing correction that can be applied under all circumstances. However, in certain special cases, more powerful procedures for multiple testing correction may be available, in order to control the FWER while achieving higher power (i.e. committing fewer Type II errors) than would be possible using Holm or Bonferroni. In this section, we have illustrated two such examples.

13.3.3 Trade-Off Between the FWER and Power

In general, there is a trade-off between the FWER threshold that we choose, and our *power* to reject the null hypotheses. Recall that power is defined as the number of false null hypotheses that we reject divided by the total number of false null hypotheses, i.e. $S/(m - m_0)$ using the notation of Table 13.2. Figure 13.5 illustrates the results of a simulation setting involving m null hypotheses, of which 90% are true and the remaining 10% are false; power is displayed as a function of the FWER. In this particular simulation setting, when $m = 10$, a FWER of 0.05 corresponds to power of approximately 60%. However, as m increases, the power decreases. With $m = 500$, the power is below 0.2 at a FWER of 0.05, so that we successfully reject only 20% of the false null hypotheses.

Figure 13.5 indicates that it is reasonable to control the FWER when m takes on a small value, like 5 or 10. However, for $m = 100$ or $m = 1,000$, attempting to control the FWER will make it almost impossible to reject any of the false null hypotheses. In other words, the power will be extremely low.

Why is this the case? Recall that, using the notation in Table 13.2, the FWER is defined as $\Pr(V \geq 1)$ (13.3). In other other words, controlling the FWER at level α guarantees that the data analyst is *very unlikely* (with probability no more than α) to reject *any* true null hypotheses, i.e. to have any false positives. In order to make good on this guarantee when m is large, the data analyst may be forced to reject very few null hypotheses, or perhaps even none at all (since if $R = 0$ then also $V = 0$; see Table 13.2).

¹⁴In fact, calculating the "correct" value of m is quite technical, and outside the scope of this book.

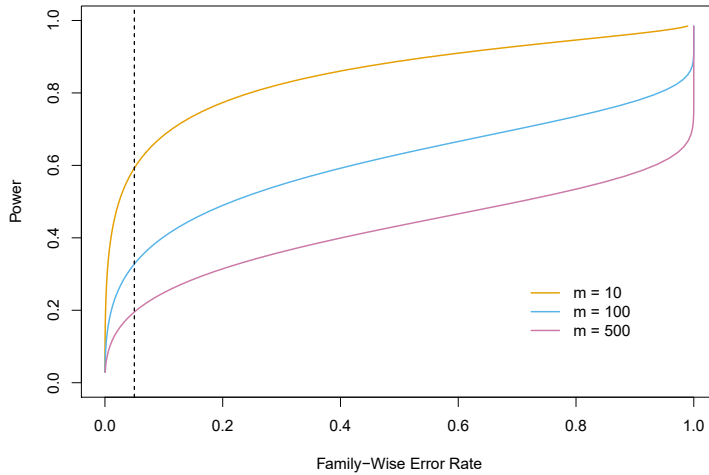


FIGURE 13.5. In a simulation setting in which 90% of the m null hypotheses are true, we display the power (the fraction of false null hypotheses that we successfully reject) as a function of the family-wise error rate. The curves correspond to $m = 10$ (orange), $m = 100$ (blue), and $m = 500$ (purple). As the value of m increases, the power decreases. The vertical dashed line indicates a FWER of 0.05.

This is scientifically uninteresting, and typically results in very low power, as in Figure 13.5.

In practice, when m is large, we may be willing to tolerate a few false positives, in the interest of making more discoveries, i.e. more rejections of the null hypothesis. This is the motivation behind the false discovery rate, which we present next.

13.4 The False Discovery Rate

13.4.1 Intuition for the False Discovery Rate

As we just discussed, when m is large, then trying to prevent *any* false positives (as in FWER control) is simply too stringent. Instead, we might try to make sure that the ratio of false positives (V) to total positives ($V + S = R$) is sufficiently low, so that most of the rejected null hypotheses are not false positives. The ratio V/R is known as the *false discovery proportion* (FDP).

false
discovery
proportion

It might be tempting to ask the data analyst to control the FDP: to make sure that no more than, say, 20% of the rejected null hypotheses are false positives. However, in practice, controlling the FDP is an impossible task for the data analyst, since she has no way to be certain, on any particular dataset, which hypotheses are true and which are false. This is very similar to the fact that the data analyst can control the FWER, i.e. she can guarantee that $\Pr(V \geq 1) \leq \alpha$ for any pre-specified α , but she cannot guarantee that $V = 0$ on any particular dataset (short of failing to reject any null hypotheses, i.e. setting $R = 0$).

Therefore, we instead control the *false discovery rate* (FDR)¹⁵, defined as

$$\text{FDR} = \text{E}(\text{FDP}) = \text{E}(V/R). \quad (13.9)$$

false
discovery
rate

When we control the FDR at (say) level $q = 20\%$, we are rejecting as many null hypotheses as possible while guaranteeing that no more than 20% of those rejected null hypotheses are false positives, *on average*.

In the definition of the FDR in (13.9), the expectation is taken over the population from which the data are generated. For instance, suppose we control the FDR for m null hypotheses at $q = 0.2$. This means that if we repeat this experiment a huge number of times, and each time control the FDR at $q = 0.2$, then we should expect that, on average, 20% of the rejected null hypotheses will be false positives. On a given dataset, the fraction of false positives among the rejected hypotheses may be greater than or less than 20%.

Thus far, we have motivated the use of the FDR from a pragmatic perspective, by arguing that when m is large, controlling the FWER is simply too stringent, and will not lead to “enough” discoveries. An additional motivation for the use of the FDR is that it aligns well with the way that data are often collected in contemporary applications. As datasets continue to grow in size across a variety of fields, it is increasingly common to conduct a huge number of hypothesis tests for exploratory, rather than confirmatory, purposes. For instance, a genomic researcher might sequence the genomes of individuals with and without some particular medical condition, and then, for each of 20,000 genes, test whether sequence variants in that gene are associated with the medical condition of interest. This amounts to performing $m = 20,000$ hypothesis tests. The analysis is exploratory in nature, in the sense that the researcher does not have any particular hypothesis in mind; instead she wishes to see whether there is modest evidence for the association between each gene and the disease, with a plan to further investigate any genes for which there is such evidence. She is likely willing to tolerate some number of false positives in the set of genes that she will investigate further; thus, the FWER is not an appropriate choice. However, some correction for multiple testing is required: it would not be a good idea for her to simply investigate *all* genes with p -values less than (say) 0.05, since we would expect 1,000 genes to have such small p -values simply by chance, even if no genes are associated with the disease (since $0.05 \times 20,000 = 1,000$). Controlling the FDR for her exploratory analysis at 20% guarantees that — on average — no more than 20% of the genes that she investigates further are false positives.

It is worth noting that unlike p -values, for which a threshold of 0.05 is typically viewed as the minimum standard of evidence for a “positive” result, and a threshold of 0.01 or even 0.001 is viewed as much more compelling, there is no standard accepted threshold for FDR control. Instead, the choice of FDR threshold is typically context-dependent, or even dataset-dependent. For instance, the genomic researcher in the previous example might seek to control the FDR at a threshold of 10% if the planned follow-

¹⁵If $R = 0$, then we replace the ratio V/R with 0, to avoid computing $0/0$. Formally, $\text{FDR} = \text{E}(V/R | R > 0) \Pr(R > 0)$.

up analysis is time-consuming or expensive. Alternatively, a much larger threshold of 30% might be suitable if she plans an inexpensive follow-up analysis.

13.4.2 The Benjamini–Hochberg Procedure

We now focus on the task of controlling the FDR: that is, deciding which null hypotheses to reject while guaranteeing that the FDR, $E(V/R)$, is less than or equal to some pre-specified value q . In order to do this, we need some way to connect the p -values, $p_1, \dots, p_m$, from the m null hypotheses to the desired FDR value, q . It turns out that a very simple procedure, outlined in Algorithm 13.2, can be used to control the FDR.

Algorithm 13.2 *Benjamini–Hochberg Procedure to Control the FDR*

1. Specify q , the level at which to control the FDR.
2. Compute p -values, $p_1, \dots, p_m$, for the m null hypotheses $H_{01}, \dots, H_{0m}$.
3. Order the m p -values so that $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.

4. Define

$$L = \max\{j : p_{(j)} < qj/m\}. \quad (13.10)$$

5. Reject all null hypotheses H_{0j} for which $p_j \leq p_{(L)}$.
-

Algorithm 13.2 is known as the *Benjamini–Hochberg procedure*. The crux of this procedure lies in (13.10). For example, consider again the first five managers in the **Fund** dataset, presented in Table 13.3. (In this example, $m = 5$, although typically we control the FDR in settings involving a much greater number of null hypotheses.) We see that $p_{(1)} = 0.006 < 0.05 \times 1/5$, $p_{(2)} = 0.012 < 0.05 \times 2/5$, $p_{(3)} = 0.601 > 0.05 \times 3/5$, $p_{(4)} = 0.756 > 0.05 \times 4/5$, and $p_{(5)} = 0.918 > 0.05 \times 5/5$. Therefore, to control the FDR at 5%, we reject the null hypotheses that the first and third fund managers perform no better than chance.

Benjamini–
Hochberg
procedure

As long as the m p -values are independent or only mildly dependent, then the Benjamini–Hochberg procedure guarantees¹⁶ that

$$\text{FDR} \leq q.$$

In other words, this procedure ensures that, on average, no more than a fraction q of the rejected null hypotheses are false positives. Remarkably, this holds regardless of how many null hypotheses are true, and regardless of the distribution of the p -values for the null hypotheses that are false. Therefore, the Benjamini–Hochberg procedure gives us a very easy way to determine, given a set of m p -values, which null hypotheses to reject in order to control the FDR at any pre-specified level q .

¹⁶However, the proof is well beyond the scope of this book.

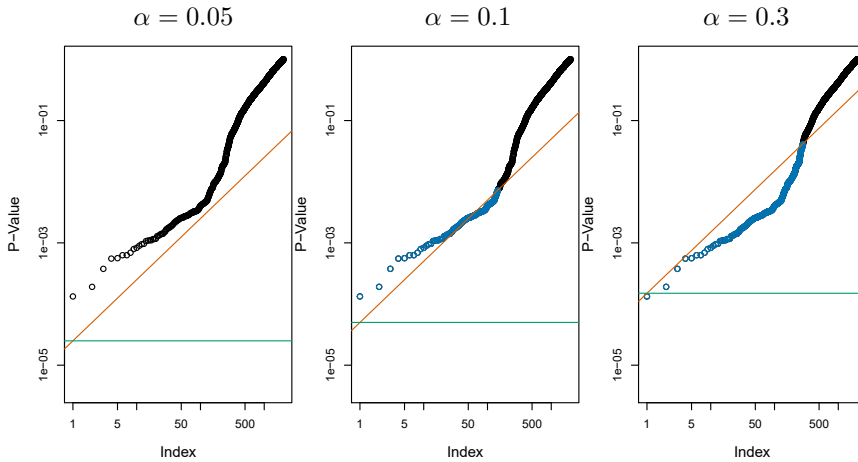


FIGURE 13.6. Each panel displays the same set of $m = 2,000$ ordered p -values for the **Fund** data. The green lines indicate the p -value thresholds corresponding to FWER control, via the Bonferroni procedure, at levels $\alpha = 0.05$ (left), $\alpha = 0.1$ (center), and $\alpha = 0.3$ (right). The orange lines indicate the p -value thresholds corresponding to FDR control, via Benjamini–Hochberg, at levels $q = 0.05$ (left), $q = 0.1$ (center), and $q = 0.3$ (right). When the FDR is controlled at level $q = 0.1$, 146 null hypotheses are rejected (center); the corresponding p -values are shown in blue. When the FDR is controlled at level $q = 0.3$, 279 null hypotheses are rejected (right); the corresponding p -values are shown in blue.

There is a fundamental difference between the Bonferroni procedure of Section 13.3.2 and the Benjamini–Hochberg procedure. In the Bonferroni procedure, in order to control the FWER for m null hypotheses at level α , we must simply reject null hypotheses for which the p -value is below α/m . This threshold of α/m does not depend on anything about the data (beyond the value of m), and certainly does not depend on the p -values themselves. By contrast, the rejection threshold used in the Benjamini–Hochberg procedure is more complicated: we reject all null hypotheses for which the p -value is less than or equal to the L th smallest p -value, where L is itself a function of all m p -values, as in (13.10). Therefore, when conducting the Benjamini–Hochberg procedure, we cannot plan out in advance what threshold we will use to reject p -values; we need to first see our data. For instance, in the abstract, there is no way to know whether we will reject a null hypothesis corresponding to a p -value of 0.01 when using an FDR threshold of 0.1 with $m = 100$; the answer depends on the values of the other $m - 1$ p -values. This property of the Benjamini–Hochberg procedure is shared by the Holm procedure, which also involves a data-dependent p -value threshold.

Figure 13.6 displays the results of applying the Bonferroni and Benjamini–Hochberg procedures on the **Fund** data set, using the full set of $m = 2,000$ fund managers, of which the first five were displayed in Table 13.3. When the FWER is controlled at level 0.3 using Bonferroni, only one null hypothesis is rejected; that is, we can conclude only that a single fund manager is beating the market. This is despite the fact that a substantial portion of

the $m = 2,000$ fund managers appear to have beaten the market without performing correction for multiple testing — for instance, 13 of them have p -values below 0.001. By contrast, when the FDR is controlled at level 0.3, we can conclude that 279 fund managers are beating the market: we expect that no more than around $279 \times 0.3 = 83.7$ of these fund managers had good performance only due to chance. Thus, we see that FDR control is much milder — and more powerful — than FWER control, in the sense that it allows us to reject many more null hypotheses, with a cost of substantially more false positives.

The Benjamini–Hochberg procedure has been around since the mid-1990s. While a great many papers have been published since then proposing alternative approaches for FDR control that can perform better in particular scenarios, the Benjamini–Hochberg procedure remains a very useful and widely-applicable approach.

13.5 A Re-Sampling Approach to p -Values and False Discovery Rates

Thus far, the discussion in this chapter has assumed that we are interested in testing a particular null hypothesis H_0 using a test statistic T , which has some known (or assumed) distribution under H_0 , such as a normal distribution, a t -distribution, a χ^2 -distribution, or an F -distribution. This is referred to as the *theoretical null distribution*. We typically rely upon the availability of a theoretical null distribution in order to obtain a p -value associated with our test statistic. Indeed, for most of the types of null hypotheses that we might be interested in testing, a theoretical null distribution is available, provided that we are willing to make stringent assumptions about our data.

theoretical
null
distribution

However, if our null hypothesis H_0 or test statistic T is somewhat unusual, then it may be the case that no theoretical null distribution is available. Alternatively, even if a theoretical null distribution exists, then we may be wary of relying upon it, perhaps because some assumption that is required for it to hold is violated. For instance, maybe the sample size is too small.

In this section, we present a framework for performing inference in this setting, which exploits the availability of fast computers in order to approximate the null distribution of T , and thereby to obtain a p -value. While this framework is very general, it must be carefully instantiated for a specific problem of interest. Therefore, in what follows, we consider a specific example in which we wish to test whether the means of two random variables are equal, using a two-sample t -test.

The discussion in this section is more challenging than the preceding sections in this chapter, and can be safely skipped by a reader who is content to use the theoretical null distribution to compute p -values for his or her test statistics.

13.5.1 A Re-Sampling Approach to the p -Value

We return to the example of Section 13.1.1, in which we wish to test whether the mean of a random variable X equals the mean of a random variable Y , i.e. $H_0 : E(X) = E(Y)$, against the alternative $H_a : E(X) \neq E(Y)$. Given n_X independent observations from X and n_Y independent observations from Y , the two-sample t -statistic takes the form

$$T = \frac{\hat{\mu}_X - \hat{\mu}_Y}{s \sqrt{\frac{1}{n_X} + \frac{1}{n_Y}}} \quad (13.11)$$

where $\hat{\mu}_X = \frac{1}{n_X} \sum_{i=1}^{n_X} x_i$, $\hat{\mu}_Y = \frac{1}{n_Y} \sum_{i=1}^{n_Y} y_i$, $s = \sqrt{\frac{(n_X-1)s_X^2 + (n_Y-1)s_Y^2}{n_X + n_Y - 2}}$, and s_X^2 and s_Y^2 are unbiased estimators of the variances in the two groups. A large (absolute) value of T provides evidence against H_0 .

If n_X and n_Y are large, then T in (13.11) approximately follows a $N(0, 1)$ distribution. But if n_X and n_Y are small, then in the absence of a strong assumption about the distribution of X and Y , we do not know the theoretical null distribution of T .¹⁷ In this case, it turns out that we can approximate the null distribution of T using a *re-sampling* approach, or more specifically, a *permutation* approach.

To do this, we conduct a thought experiment. If H_0 holds, so that $E(X) = E(Y)$, and we make the stronger assumption that the distributions of X and Y are the same, then the distribution of T is invariant under swapping observations of X with observations of Y . That is, if we randomly swap some of the observations in X with the observations in Y , then *the test statistic T in (13.11) computed based on this swapped data has the same distribution as T based on the original data.* This is true only if H_0 holds, and the distributions of X and Y are the same.

This suggests that in order to approximate the null distribution of T , we can take the following approach. We randomly permute the $n_X + n_Y$ observations B times, for some large value of B , and each time we compute (13.11). We let $T^{*1}, \dots, T^{*B}$ denote the values of (13.11) on the permuted data. These can be viewed as an approximation of the null distribution of T under H_0 . Recall that by definition, a p -value is the probability of observing a test statistic at least this extreme under H_0 . Therefore, to compute a p -value for T , we can simply compute

$$p\text{-value} = \frac{\sum_{b=1}^B 1_{(|T^{*b}| \geq |T|)}}{B}, \quad (13.12)$$

the fraction of permuted datasets for which the value of the test statistic is at least as extreme as the value observed on the original data. This procedure is summarized in Algorithm 13.3.

¹⁷If we assume that X and Y are normally distributed, then T in (13.11) follows a t -distribution with $n_X + n_Y - 2$ degrees of freedom under H_0 . However, in practice, the distribution of random variables is rarely known, and so it can be preferable to perform a re-sampling approach instead of making strong and unjustified assumptions. If the results of the re-sampling approach disagree with the results of assuming a theoretical null distribution, then the results of the re-sampling approach are more trustworthy.

Algorithm 13.3 *Re-Sampling p -Value for a Two-Sample t -Test*

1. Compute T , defined in (13.11), on the original data $x_1, \dots, x_{n_X}$ and $y_1, \dots, y_{n_Y}$.
2. For $b = 1, \dots, B$, where B is a large number (e.g. $B = 10,000$):
 - (a) Permute the $n_X + n_Y$ observations at random. Call the first n_X permuted observations $x_1^*, \dots, x_{n_X}^*$, and call the remaining n_Y observations $y_1^*, \dots, y_{n_Y}^*$.
 - (b) Compute (13.11) on the permuted data $x_1^*, \dots, x_{n_X}^*$ and $y_1^*, \dots, y_{n_Y}^*$, and call the result T^{*b} .
3. The p -value is given by $\frac{\sum_{b=1}^B 1_{(|T^{*b}| \geq |T|)}}{B}$.

We try out this procedure on the **Khan** dataset, which consists of expression measurements for 2,308 genes in four sub-types of small round blood cell tumors, a type of cancer typically seen in children. This dataset is part of the **ISLR2** package. We restrict our attention to the two sub-types for which the most observations are available: rhabdomyosarcoma ($n_X = 29$) and Burkitt's lymphoma ($n_Y = 25$).

A two-sample t -test for the null hypothesis that the 11th gene's mean expression values are equal in the two groups yields $T = -2.09$. Using the theoretical null distribution, which is a t_{52} distribution (since $n_X + n_Y - 2 = 52$), we obtain a p -value of 0.041. (Note that a t_{52} distribution is virtually indistinguishable from a $N(0, 1)$ distribution.) If we instead apply Algorithm 13.3 with $B = 10,000$, then we obtain a p -value of 0.042. Figure 13.7 displays the theoretical null distribution, the re-sampling null distribution, and the actual value of the test statistic ($T = -2.09$) for this gene. In this example, we see very little difference between the p -values obtained using the theoretical null distribution and the re-sampling null distribution.

By contrast, Figure 13.8 shows an analogous set of results for the 877th gene. In this case, there is a substantial difference between the theoretical and re-sampling null distributions, which results in a difference between their p -values.

In general, in settings with a smaller sample size or a more skewed data distribution (so that the theoretical null distribution is less accurate), the difference between the re-sampling and theoretical p -values will tend to be more pronounced. In fact, the substantial difference between the re-sampling and theoretical null distributions in Figure 13.8 is due to the fact that a single observation in the 877th gene is very far from the other observations, leading to a very skewed distribution.

13.5.2 A Re-Sampling Approach to the False Discovery Rate

Now, suppose that we wish to control the FDR for m null hypotheses, $H_{01}, \dots, H_{0m}$, in a setting in which either no theoretical null distribution is available, or else we simply prefer to avoid the use of a theoretical null



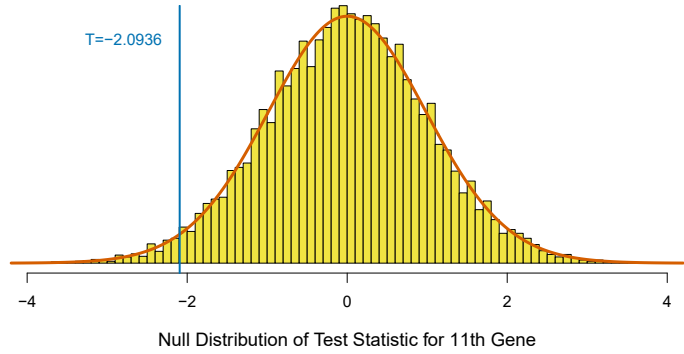


FIGURE 13.7. The 11th gene in the *Khan* dataset has a test statistic of $T = -2.09$. Its theoretical and re-sampling null distributions are almost identical. The theoretical p -value equals 0.041 and the re-sampling p -value equals 0.042.

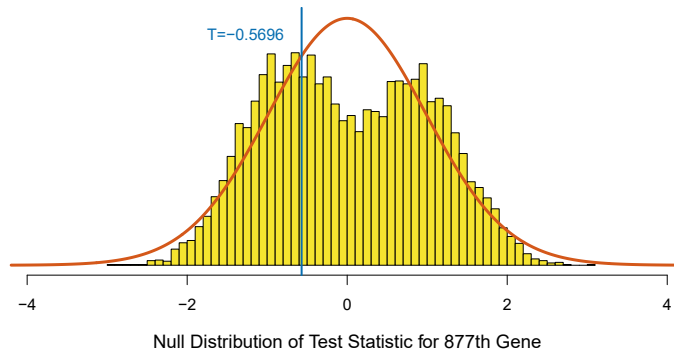


FIGURE 13.8. The 877th gene in the *Khan* dataset has a test statistic of $T = -0.57$. Its theoretical and re-sampling null distributions are quite different. The theoretical p -value equals 0.571, and the re-sampling p -value equals 0.673.

distribution. As in Section 13.5.1, we make use of a two-sample t -statistic for each hypothesis, leading to the test statistics $T_1, \dots, T_m$. We could simply compute a p -value for each of the m null hypotheses, as in Section 13.5.1, and then apply the Benjamini–Hochberg procedure of Section 13.4.2 to these p -values. However, it turns out that we can do this in a more direct way, without even needing to compute p -values.

Recall from Section 13.4 that the FDR is defined as $E(V/R)$, using the notation in Table 13.2. In order to estimate the FDR via re-sampling, we first make the following approximation:

$$\text{FDR} = E\left(\frac{V}{R}\right) \approx \frac{E(V)}{R}. \quad (13.13)$$

Now suppose we reject any null hypothesis for which the test statistic exceeds c in absolute value. Then computing R in the denominator on the right-hand side of (13.13) is straightforward: $R = \sum_{j=1}^m 1_{(|T_j| \geq c)}$.

However, the numerator $E(V)$ on the right-hand side of (13.13) is more challenging. This is the expected number of false positives associated with rejecting any null hypothesis for which the test statistic exceeds c in absolute value. At the risk of stating the obvious, estimating V is challenging because we do not know which of $H_{01}, \dots, H_{0m}$ are really true, and so we do not know which rejected hypotheses are false positives. To overcome this problem, we take a re-sampling approach, in which we simulate data under $H_{01}, \dots, H_{0m}$, and then compute the resulting test statistics. The number of re-sampled test statistics that exceed c provides an estimate of V .

In greater detail, in the case of a two-sample t -statistic (13.11) for each of the null hypotheses $H_{01}, \dots, H_{0m}$, we can estimate $E(V)$ as follows. Let $x_1^{(j)}, \dots, x_{n_X}^{(j)}$ and $y_1^{(j)}, \dots, y_{n_Y}^{(j)}$ denote the data associated with the j th null hypothesis, $j = 1, \dots, m$. We permute these $n_X + n_Y$ observations at random, and then compute the t -statistic on the permuted data. For this permuted data, we know that all of the null hypotheses $H_{01}, \dots, H_{0m}$ hold; therefore, the number of permuted t -statistics that exceed the threshold c in absolute value provides an estimate for $E(V)$. This estimate can be further improved by repeating the permutation process B times, for a large value of B , and averaging the results.

Algorithm 13.4 details this procedure.¹⁸ It provides what is known as a *plug-in estimate* of the FDR, because the approximation in (13.13) allows us to estimate the FDR by plugging R into the denominator and an estimate for $E(V)$ into the numerator.

We apply the re-sampling approach to the FDR from Algorithm 13.4, as well as the Benjamini–Hochberg approach from Algorithm 13.2 using theoretical p -values, to the $m = 2,308$ genes in the **Khan** dataset. Results are shown in Figure 13.9. We see that for a given number of rejected hypotheses, the estimated FDRs are almost identical for the two methods.

We began this section by noting that in order to control the FDR for m hypothesis tests using a re-sampling approach, we could simply compute m re-sampling p -values as in Section 13.5.1, and then apply the Benjamini–Hochberg procedure of Section 13.4.2 to these p -values. It turns out that if we define the j th re-sampling p -value as

$$p_j = \frac{\sum_{j'=1}^m \sum_{b=1}^B 1_{(|T_{j'}^{*b}| \geq |T_j|)}}{Bm} \quad (13.14)$$

for $j = 1, \dots, m$, instead of as in (13.12), then applying the Benjamini–Hochberg procedure to these re-sampled p -values is *exactly* equivalent to Algorithm 13.4. Note that (13.14) is an alternative to (13.12) that pools the information across all m hypothesis tests in approximating the null distribution.

13.5.3 When Are Re-Sampling Approaches Useful?

In Sections 13.5.1 and 13.5.2, we considered testing null hypotheses of the form $H_0 : E(X) = E(Y)$ using a two-sample t -statistic (13.11), for which we

¹⁸To implement Algorithm 13.4 efficiently, the same set of permutations in Step 2(b)i. should be used for all m null hypotheses.

Algorithm 13.4 *Plug-In FDR for a Two-Sample T-Test*

1. Select a threshold c , where $c > 0$.
2. For $j = 1, \dots, m$:
 - (a) Compute $T^{(j)}$, the two-sample t -statistic (13.11) for the null hypothesis H_{0j} on the basis of the original data, $x_1^{(j)}, \dots, x_{n_X}^{(j)}$ and $y_1^{(j)}, \dots, y_{n_Y}^{(j)}$.
 - (b) For $b = 1, \dots, B$, where B is a large number (e.g. $B = 10,000$):
 - i. Permute the $n_X + n_Y$ observations at random. Call the first n_X observations $x_1^{*(j)}, \dots, x_{n_X}^{*(j)}$, and call the remaining observations $y_1^{*(j)}, \dots, y_{n_Y}^{*(j)}$.
 - ii. Compute (13.11) on the permuted data $x_1^{*(j)}, \dots, x_{n_X}^{*(j)}$ and $y_1^{*(j)}, \dots, y_{n_Y}^{*(j)}$, and call the result $T^{(j),*b}$.
3. Compute $R = \sum_{j=1}^m 1_{(|T^{(j)}| \geq c)}$.
4. Compute $\hat{V} = \frac{\sum_{b=1}^B \sum_{j=1}^m 1_{(|T^{(j),*b}| \geq c)}}{B}$.
5. The estimated FDR associated with the threshold c is $\hat{V}/R$.

approximated the null distribution via a re-sampling approach. We saw that using the re-sampling approach gave us substantially different results from using the theoretical p -value approach in Figure 13.8, but not in Figure 13.7.

In general, there are two settings in which a re-sampling approach is particularly useful:

1. Perhaps no theoretical null distribution is available. This may be the case if you are testing an unusual null hypothesis H_0 , or using an unusual test statistic T .
2. Perhaps a theoretical null distribution *is* available, but the assumptions required for its validity do not hold. For instance, the two-sample t -statistic in (13.11) follows a $t_{n_X+n_Y-2}$ distribution only if the observations are normally distributed. Furthermore, it follows a $N(0, 1)$ distribution only if n_X and n_Y are quite large. If the data are non-normal and n_X and n_Y are small, then p -values that make use of the theoretical null distribution will not be valid (i.e. they will not properly control the Type I error).

In general, if you can come up with a way to re-sample or permute your observations in order to generate data that follow the null distribution, then you can compute p -values or estimate the FDR using variants of Algorithms 13.3 and 13.4. In many real-world settings, this provides a powerful tool for hypothesis testing when no out-of-box hypothesis tests are available, or when the key assumptions underlying those out-of-box tests are violated.

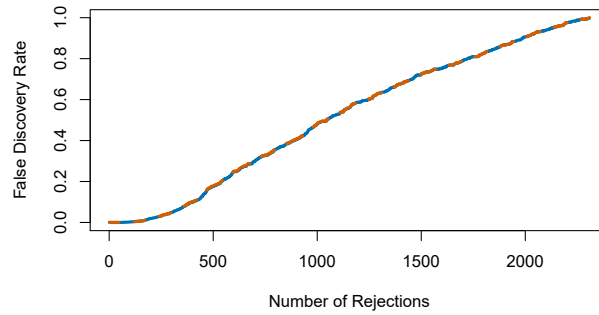


FIGURE 13.9. For $j = 1, \dots, m = 2,308$, we tested the null hypothesis that for the j th gene in the **Khan** dataset, the mean expression in Burkitt's lymphoma equals the mean expression in rhabdomyosarcoma. For each value of k from 1 to 2,308, the y -axis displays the estimated FDR associated with rejecting the null hypotheses corresponding to the k smallest p -values. The orange dashed curve shows the FDR obtained using the Benjamini–Hochberg procedure, whereas the blue solid curve shows the FDR obtained using the re-sampling approach of Algorithm 13.4, with $B = 10,000$. There is very little difference between the two FDR estimates. According to either estimate, rejecting the null hypothesis for the 500 genes with the smallest p -values corresponds to an FDR of around 17.7%.

13.6 Lab: Multiple Testing

We include our usual imports seen in earlier labs.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from ISLP import load_data
```

We also collect the new imports needed for this lab.

```
In [2]: from scipy.stats import \
        (ttest_isamp,
         ttest_rel,
         ttest_ind,
         t as t_dbn)
from statsmodels.stats.multicomp import \
        pairwise_tukeyhsd
from statsmodels.stats.multitest import \
        multipletests as mult_test
```

13.6.1 Review of Hypothesis Tests

We begin by performing some one-sample t -tests. First we create 100 variables, each consisting of 10 observations. The first 50 variables have mean 0.5 and variance 1, while the others have mean 0 and variance 1.

```
In [3]: rng = np.random.default_rng(12)
X = rng.standard_normal((10, 100))
true_mean = np.array([0.5]*50 + [0]*50)
X += true_mean[None,:]
```

To begin, we use `ttest_1samp()` from the `scipy.stats` module to test $H_0 : \mu_1 = 0$, the null hypothesis that the first variable has mean zero. `ttest_1samp()`

```
In [4]: result = ttest_1samp(X[:,0], 0)
        result.pvalue
```

```
Out[4]: 0.931
```

The p -value comes out to 0.931, which is not low enough to reject the null hypothesis at level $\alpha = 0.05$. In this case, $\mu_1 = 0.5$, so the null hypothesis is false. Therefore, we have made a Type II error by failing to reject the null hypothesis when the null hypothesis is false.

We now test $H_{0,j} : \mu_j = 0$ for $j = 1, \dots, 100$. We compute the 100 p -values, and then construct a vector recording whether the j th p -value is less than or equal to 0.05, in which case we reject $H_{0,j}$, or greater than 0.05, in which case we do not reject $H_{0,j}$, for $j = 1, \dots, 100$.

```
In [5]: p_values = np.empty(100)
        for i in range(100):
            p_values[i] = ttest_1samp(X[:,i], 0).pvalue
        decision = pd.cut(p_values,
                           [0, 0.05, 1],
                           labels=['Reject H0',
                                   'Do not reject H0'])
        truth = pd.Categorical(true_mean == 0,
                               categories=[True, False],
                               ordered=True)
```

Since this is a simulated data set, we can create a 2×2 table similar to Table 13.2.

```
In [6]: pd.crosstab(decision,
                    truth,
                    rownames=['Decision'],
                    colnames=['H0'])
```

```
Out[6]:
```

	H0	True	False
Decision			
Reject H0		5	15
Do not reject H0		45	35

Therefore, at level $\alpha = 0.05$, we reject 15 of the 50 false null hypotheses, and we incorrectly reject 5 of the true null hypotheses. Using the notation from Section 13.3, we have $V = 5$, $S = 15$, $U = 45$ and $W = 35$. We have set $\alpha = 0.05$, which means that we expect to reject around 5% of the true null hypotheses. This is in line with the 2×2 table above, which indicates that we rejected $V = 5$ of the 50 true null hypotheses.

In the simulation above, for the false null hypotheses, the ratio of the mean to the standard deviation was only $0.5/1 = 0.5$. This amounts to quite a weak signal, and it resulted in a high number of Type II errors. Let's instead simulate data with a stronger signal, so that the ratio of the mean to the standard deviation for the false null hypotheses equals 1. We make only 10 Type II errors.

```
In [7]: true_mean = np.array([1]*50 + [0]*50)
X = rng.standard_normal((10, 100))
X += true_mean[None,:]
for i in range(100):
    p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
                  [0, 0.05, 1],
                  labels=['Reject H0',
                          'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
                       categories=[True, False],
                       ordered=True)
pd.crosstab(decision,
            truth,
            rownames=['Decision'],
            colnames=['H0'])
```

```
Out[7]:
```

	H0	True	False
Decision			
Reject H0		2	40
Do not reject H0		48	10

13.6.2 Family-Wise Error Rate

Recall from (13.5) that if the null hypothesis is true for each of m independent hypothesis tests, then the FWER is equal to $1 - (1 - \alpha)^m$. We can use this expression to compute the FWER for $m = 1, \dots, 500$ and $\alpha = 0.05, 0.01$, and 0.001 . We plot the FWER for these values of α in order to reproduce Figure 13.2.

```
In [8]: m = np.linspace(1, 501)
fig, ax = plt.subplots()
[ax.plot(m,
         1 - (1 - alpha)**m,
         label=r'$\alpha=%s$' % str(alpha))
 for alpha in [0.05, 0.01, 0.001]]
ax.set_xscale('log')
ax.set_xlabel('Number of Hypotheses')
ax.set_ylabel('Family-Wise Error Rate')
ax.legend()
ax.axhline(0.05, c='k', ls='--');
```

As discussed previously, even for moderate values of m such as 50, the FWER exceeds 0.05 unless α is set to a very low value, such as 0.001. Of course, the problem with setting α to such a low value is that we are likely to make a number of Type II errors: in other words, our power is very low.

We now conduct a one-sample t -test for each of the first five managers in the `Fund` dataset, in order to test the null hypothesis that the j th fund manager's mean return equals zero, $H_{0,j} : \mu_j = 0$.

```
In [9]: Fund = load_data('Fund')
fund_mini = Fund.iloc[:, :5]
fund_mini_pvals = np.empty(5)
for i in range(5):
```

```
fund_mini_pvals[i] = ttest_1samp(fund_mini.iloc[:,i], 0).pvalue
fund_mini_pvals
```

```
Out[9]: array([0.006, 0.918, 0.012, 0.601, 0.756])
```

The p -values are low for Managers One and Three, and high for the other three managers. However, we cannot simply reject $H_{0,1}$ and $H_{0,3}$, since this would fail to account for the multiple testing that we have performed. Instead, we will conduct Bonferroni's method and Holm's method to control the FWER.

To do this, we use the `multipletests()` function from the `statsmodels` module (abbreviated to `mult_test()`). Given the p -values, for methods like Holm and Bonferroni the function outputs *adjusted p -values*, which can be thought of as a new set of p -values that have been corrected for multiple testing. If the adjusted p -value for a given hypothesis is less than or equal to α , then that hypothesis can be rejected while maintaining a FWER of no more than α . In other words, for such methods, the adjusted p -values resulting from the `multipletests()` function can simply be compared to the desired FWER in order to determine whether or not to reject each hypothesis. We will later see that we can use the same function to control FDR as well.

`multipletests()`
adjusted
 p -values

The `mult_test()` function takes p -values and a `method` argument, as well as an optional `alpha` argument. It returns the decisions (`reject` below) as well as the adjusted p -values (`bonf`).

```
In [10]: reject, bonf = mult_test(fund_mini_pvals, method = "bonferroni")[:2]
reject
```

```
Out[10]: array([ True, False, False, False, False])
```

The p -values `bonf` are simply the `fund_mini_pvalues` multiplied by 5 and truncated to be less than or equal to 1.

```
In [11]: bonf, np.minimum(fund_mini_pvals * 5, 1)
```

```
Out[11]: (array([0.03, 1.   , 0.06, 1.   , 1.   ]),
          array([0.03, 1.   , 0.06, 1.   , 1.   ]))
```

Therefore, using Bonferroni's method, we are able to reject the null hypothesis only for Manager One while controlling FWER at 0.05.

By contrast, using Holm's method, the adjusted p -values indicate that we can reject the null hypotheses for Managers One and Three at a FWER of 0.05.

```
In [12]: mult_test(fund_mini_pvals, method = "holm", alpha=0.05)[:2]
```

```
Out[12]: (array([ True, False,  True, False, False]),
          array([0.03, 1.   , 0.05, 1.   , 1.   ]))
```

As discussed previously, Manager One seems to perform particularly well, whereas Manager Two has poor performance.

```
In [13]: fund_mini.mean()
```

```
Out[13]: Manager1      3.0
         Manager2     -0.1
         Manager3      2.8
         Manager4      0.5
         Manager5      0.3
         dtype: float64
```

Is there evidence of a meaningful difference in performance between these two managers? We can check this by performing a *paired t-test* using the `ttest_rel()` function from `scipy.stats`:

paired *t*-test
`ttest_rel()`

```
In [14]: ttest_rel(fund_mini['Manager1'],
                  fund_mini['Manager2']).pvalue
```

```
Out[14]: 0.038
```

The test results in a *p*-value of 0.038, suggesting a statistically significant difference.

However, we decided to perform this test only after examining the data and noting that Managers One and Two had the highest and lowest mean performances. In a sense, this means that we have implicitly performed $\binom{5}{2} = 5(5-1)/2 = 10$ hypothesis tests, rather than just one, as discussed in Section 13.3.2. Hence, we use the `pairwise_tukeyhsd()` function from `statsmodels.stats.multicomp` to apply Tukey's method in order to adjust for multiple testing. This function takes as input a fitted ANOVA regression model, which is essentially just a linear regression in which all of the predictors are qualitative. In this case, the response consists of the monthly excess returns achieved by each manager, and the predictor indicates the manager to which each return corresponds.

pairwise_
tukeyhsd()
ANOVA

```
In [15]: returns = np.hstack([fund_mini.iloc[:,i] for i in range(5)])
         managers = np.hstack([[i+1]*50 for i in range(5)])
         tukey = pairwise_tukeyhsd(returns, managers)
         print(tukey.summary())
```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj   lower  upper  reject
-----
      1      2      -3.1 0.1862 -6.9865  0.7865  False
      1      3      -0.2 0.9999 -4.0865  3.6865  False
      1      4      -2.5 0.3948 -6.3865  1.3865  False
      1      5      -2.7 0.3152 -6.5865  1.1865  False
      2      3       2.9 0.2453 -0.9865  6.7865  False
      2      4       0.6 0.9932 -3.2865  4.4865  False
      2      5       0.4 0.9986 -3.4865  4.2865  False
      3      4      -2.3 0.482  -6.1865  1.5865  False
      3      5      -2.5 0.3948 -6.3865  1.3865  False
      4      5      -0.2 0.9999 -4.0865  3.6865  False
=====
```

The `pairwise_tukeyhsd()` function provides confidence intervals for the difference between each pair of managers (`lower` and `upper`), as well as a

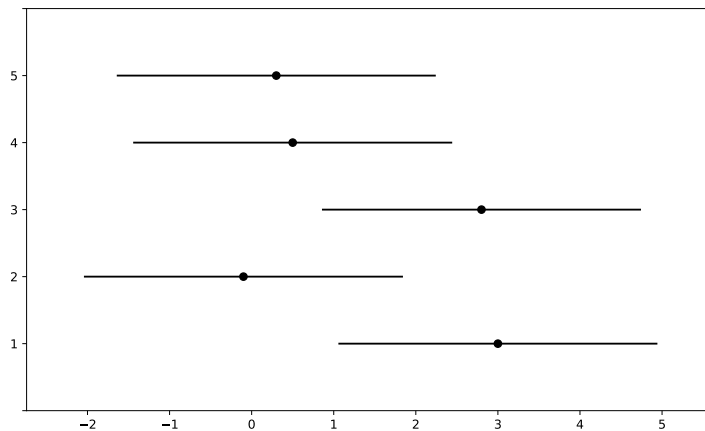


FIGURE 13.10. 95% confidence intervals for each manager on the `Fund` data, using Tukey’s method to adjust for multiple testing. All of the confidence intervals overlap, so none of the differences among managers are statistically significant when controlling FWER at level 0.05.

p -value. All of these quantities have been adjusted for multiple testing. Notice that the p -value for the difference between Managers One and Two has increased from 0.038 to 0.186, so there is no longer clear evidence of a difference between the managers’ performances. We can plot the confidence intervals for the pairwise comparisons using the `plot_simultaneous()` method of `tukey`. Any pair of intervals that don’t overlap indicates a significant difference at the nominal level of 0.05. In this case, no differences are considered significant as reported in the table above.

```
In [16]: fig, ax = plt.subplots(figsize=(8,8))
         tukey.plot_simultaneous(ax=ax);
```

The result can be seen¹⁹ in Figure 13.10.

13.6.3 False Discovery Rate

Now we perform hypothesis tests for all 2,000 fund managers in the `Fund` dataset. We perform a one-sample t -test of $H_{0,j} : \mu_j = 0$, which states that the j th fund manager’s mean return is zero.

```
In [17]: fund_pvalues = np.empty(2000)
         for i, manager in enumerate(Fund.columns):
             fund_pvalues[i] = ttest_1samp(Fund[manager], 0).pvalue
```

There are far too many managers to consider trying to control the FWER. Instead, we focus on controlling the FDR: that is, the expected fraction of rejected null hypotheses that are actually false positives. The

¹⁹Traditionally this plot shows intervals for each paired difference. With many groups it is more convenient and equivalent to display one interval per group, as is done here. By “differencing” all pairs of intervals displayed here you recover the traditional plot.

`multipletests()` function (abbreviated `mult_test()`) can be used to carry out the Benjamini–Hochberg procedure.

```
In [18]: fund_qvalues = mult_test(fund_pvalues, method = "fdr_bh")[1]
         fund_qvalues[:10]
```

```
Out[18]: array([0.09, 0.99, 0.12, 0.92, 0.96, 0.08, 0.08, 0.08, 0.08,
               0.08])
```

The *q-values* output by the Benjamini–Hochberg procedure can be interpreted as the smallest FDR threshold at which we would reject a particular null hypothesis. For instance, a *q*-value of 0.1 indicates that we can reject the corresponding null hypothesis at an FDR of 10% or greater, but that we cannot reject the null hypothesis at an FDR below 10%.

If we control the FDR at 10%, then for how many of the fund managers can we reject $H_{0,j} : \mu_j = 0$?

```
In [19]: (fund_qvalues <= 0.1).sum()
```

```
Out[19]: 146
```

We find that 146 of the 2,000 fund managers have a *q*-value below 0.1; therefore, we are able to conclude that 146 of the fund managers beat the market at an FDR of 10%. Only about 15 (10% of 146) of these fund managers are likely to be false discoveries.

By contrast, if we had instead used Bonferroni’s method to control the FWER at level $\alpha = 0.1$, then we would have failed to reject any null hypotheses!

```
In [20]: (fund_pvalues <= 0.1 / 2000).sum()
```

```
Out[20]: 0
```

Figure 13.6 displays the ordered *p*-values, $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(2000)}$, for the `Fund` dataset, as well as the threshold for rejection by the Benjamini–Hochberg procedure. Recall that the Benjamini–Hochberg procedure identifies the largest *p*-value such that $p_{(j)} < qj/m$, and rejects all hypotheses for which the *p*-value is less than or equal to $p_{(j)}$. In the code below, we implement the Benjamini–Hochberg procedure ourselves, in order to illustrate how it works. We first order the *p*-values. We then identify all *p*-values that satisfy $p_{(j)} < qj/m$ (`sorted_set_`). Finally, `selected_` is a boolean array indicating which *p*-values are less than or equal to the largest *p*-value in `sorted_[sorted_set_]`. Therefore, `selected_` indexes the *p*-values rejected by the Benjamini–Hochberg procedure.

```
In [21]: sorted_ = np.sort(fund_pvalues)
         m = fund_pvalues.shape[0]
         q = 0.1
         sorted_set_ = np.where(sorted_ < q * np.linspace(1, m, m) / m)[0]
         if sorted_set_.shape[0] > 0:
             selected_ = fund_pvalues < sorted_[sorted_set_].max()
             sorted_set_ = np.arange(sorted_set_.max())
         else:
             selected_ = []
             sorted_set_ = []
```

We now reproduce the middle panel of Figure 13.6.

```
In [22]: fig, ax = plt.subplots()
ax.scatter(np.arange(0, sorted_.shape[0]) + 1,
           sorted_, s=10)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_ylabel('P-Value')
ax.set_xlabel('Index')
ax.scatter(sorted_set_+1, sorted_[sorted_set_], c='r', s=20)
ax.axline((0, 0), (1,q/m), c='k', ls='--', linewidth=3);
```

13.6.4 A Re-Sampling Approach

Here, we implement the re-sampling approach to hypothesis testing using the *Khan* dataset, which we investigated in Section 13.5. First, we merge the training and testing data, which results in observations on 83 patients for 2,308 genes.

```
In [23]: Khan = load_data('Khan')
D = pd.concat([Khan['xtrain'], Khan['xtest']])
D['Y'] = pd.concat([Khan['ytrain'], Khan['ytest']])
D['Y'].value_counts()
```

```
Out [23]: 2    29
         4    25
         3    18
         1    11
         Name: Y, dtype: int64
```

There are four classes of cancer. For each gene, we compare the mean expression in the second class (rhabdomyosarcoma) to the mean expression in the fourth class (Burkitt's lymphoma). Performing a standard two-sample *t*-test using `ttest_ind()` from `scipy.stats` on the 11th gene produces a test-statistic of -2.09 and an associated *p*-value of 0.0412, suggesting modest evidence of a difference in mean expression levels between the two cancer types.

`ttest_ind()`

```
In [24]: D2 = D[lambda df:df['Y'] == 2]
D4 = D[lambda df:df['Y'] == 4]
gene_11 = 'G0011'
observedT, pvalue = ttest_ind(D2[gene_11],
                             D4[gene_11],
                             equal_var=True)
observedT, pvalue
```

```
Out [24]: (-2.094, 0.041)
```

However, this *p*-value relies on the assumption that under the null hypothesis of no difference between the two groups, the test statistic follows a *t*-distribution with $29 + 25 - 2 = 52$ degrees of freedom. Instead of using this theoretical null distribution, we can randomly split the 54 patients into two groups of 29 and 25, and compute a new test statistic. Under the null hypothesis of no difference between the groups, this new test statistic should have the same distribution as our original one. Repeating this

process 10,000 times allows us to approximate the null distribution of the test statistic. We compute the fraction of the time that our observed test statistic exceeds the test statistics obtained via re-sampling.

```
In [25]: B = 10000
Tnull = np.empty(B)
D_ = np.hstack([D2[gene_11], D4[gene_11]])
n_ = D2[gene_11].shape[0]
D_null = D_.copy()
for b in range(B):
    rng.shuffle(D_null)
    ttest_ = ttest_ind(D_null[:n_],
                      D_null[n_:],
                      equal_var=True)
    Tnull[b] = ttest_.statistic
(np.abs(Tnull) > np.abs(observedT)).mean()
```

```
Out[25]: 0.0398
```

This fraction, 0.0398, is our re-sampling-based p -value. It is almost identical to the p -value of 0.0412 obtained using the theoretical null distribution. We can plot a histogram of the re-sampling-based test statistics in order to reproduce Figure 13.7.

```
In [26]: fig, ax = plt.subplots(figsize=(8,8))
ax.hist(Tnull,
        bins=100,
        density=True,
        facecolor='y',
        label='Null')
xval = np.linspace(-4.2, 4.2, 1001)
ax.plot(xval,
        t_dbn.pdf(xval, D_.shape[0]-2),
        c='r')
ax.axvline(observedT,
           c='b',
           label='Observed')
ax.legend()
ax.set_xlabel("Null Distribution of Test Statistic");
```

The re-sampling-based null distribution is almost identical to the theoretical null distribution, which is displayed in red.

Finally, we implement the plug-in re-sampling FDR approach outlined in Algorithm 13.4. Depending on the speed of your computer, calculating the FDR for all 2,308 genes in the *Khan* dataset may take a while. Hence, we will illustrate the approach on a random subset of 100 genes. For each gene, we first compute the observed test statistic, and then produce 10,000 re-sampled test statistics. This may take a few minutes to run. If you are in a rush, then you could set `B` equal to a smaller value (e.g. `B=500`).

```
In [27]: m, B = 100, 10000
idx = rng.choice(Khan['xtest'].columns, m, replace=False)
T_vals = np.empty(m)
Tnull_vals = np.empty((m, B))

for j in range(m):
    col = idx[j]
```

```

T_vals[j] = ttest_ind(D2[col],
                     D4[col],
                     equal_var=True).statistic
D_ = np.hstack([D2[col], D4[col]])
D_null = D_.copy()
for b in range(B):
    rng.shuffle(D_null)
    ttest_ = ttest_ind(D_null[:n_],
                     D_null[n_:],
                     equal_var=True)
    Tnull_vals[j,b] = ttest_.statistic

```

Next, we compute the number of rejected null hypotheses R , the estimated number of false positives $\hat{V}$, and the estimated FDR, for a range of threshold values c in Algorithm 13.4. The threshold values are chosen using the absolute values of the test statistics from the 100 genes.

```

In [28]: cutoffs = np.sort(np.abs(T_vals))
        FDRs, Rs, Vs = np.empty((3, m))
        for j in range(m):
            R = np.sum(np.abs(T_vals) >= cutoffs[j])
            V = np.sum(np.abs(Tnull_vals) >= cutoffs[j]) / B
            Rs[j] = R
            Vs[j] = V
            FDRs[j] = V / R

```

Now, for any given FDR, we can find the genes that will be rejected. For example, with FDR controlled at 0.1, we reject 15 of the 100 null hypotheses. On average, we would expect about one or two of these genes (i.e. 10% of 15) to be false discoveries. At an FDR of 0.2, we can reject the null hypothesis for 28 genes, of which we expect around six to be false discoveries.

The variable `idx` stores which genes were included in our 100 randomly-selected genes. Let's look at the genes whose estimated FDR is less than 0.1.

```

In [29]: sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.1].min()])

```

At an FDR threshold of 0.2, more genes are selected, at the cost of having a higher expected proportion of false discoveries.

```

In [30]: sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.2].min()])

```

The next line generates Figure 13.11, which is similar to Figure 13.9, except that it is based on only a subset of the genes.

```

In [31]: fig, ax = plt.subplots()
        ax.plot(Rs, FDRs, 'b', linewidth=3)
        ax.set_xlabel("Number of Rejections")
        ax.set_ylabel("False Discovery Rate");

```

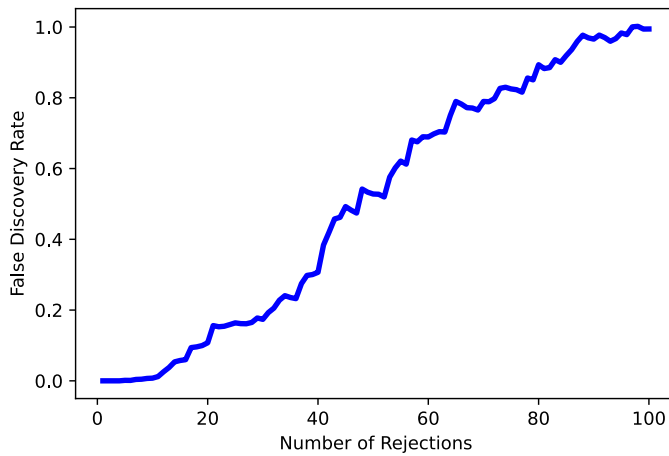


FIGURE 13.11. The estimated false discovery rate versus the number of rejected null hypotheses, for 100 genes randomly selected from the **Khan** dataset.

13.7 Exercises

Conceptual

- Suppose we test m null hypotheses, all of which are true. We control the Type I error for each null hypothesis at level α . For each subproblem, justify your answer.

- In total, how many Type I errors do we expect to make?
- Suppose that the m tests that we perform are independent. What is the family-wise error rate associated with these m tests?
Hint: If two events A and B are independent, then $\Pr(A \cap B) = \Pr(A) \Pr(B)$.
- Suppose that $m = 2$, and that the p -values for the two tests are positively correlated, so that if one is small then the other will tend to be small as well, and if one is large then the other will tend to be large. How does the family-wise error rate associated with these $m = 2$ tests qualitatively compare to the answer in (b) with $m = 2$?

Hint: First, suppose that the two p -values are perfectly correlated.

- Suppose again that $m = 2$, but that now the p -values for the two tests are negatively correlated, so that if one is large then the other will tend to be small. How does the family-wise error rate associated with these $m = 2$ tests qualitatively compare to the answer in (b) with $m = 2$?

Hint: First, suppose that whenever one p -value is less than α , then the other will be greater than α . In other words, we can never reject both null hypotheses.

2. Suppose that we test m hypotheses, and control the Type I error for each hypothesis at level α . Assume that all m p -values are independent, and that all null hypotheses are true.
 - (a) Let the random variable A_j equal 1 if the j th null hypothesis is rejected, and 0 otherwise. What is the distribution of A_j ?
 - (b) What is the distribution of $\sum_{j=1}^m A_j$?
 - (c) What is the standard deviation of the number of Type I errors that we will make?
3. Suppose we test m null hypotheses, and control the Type I error for the j th null hypothesis at level α_j , for $j = 1, \dots, m$. Argue that the family-wise error rate is no greater than $\sum_{j=1}^m \alpha_j$.

Null Hypothesis	p -value
H_{01}	0.0011
H_{02}	0.031
H_{03}	0.017
H_{04}	0.32
H_{05}	0.11
H_{06}	0.90
H_{07}	0.07
H_{08}	0.006
H_{09}	0.004
H_{10}	0.0009

TABLE 13.4. p -values for Exercise 4.

4. Suppose we test $m = 10$ hypotheses, and obtain the p -values shown in Table 13.4.
 - (a) Suppose that we wish to control the Type I error for each null hypothesis at level $\alpha = 0.05$. Which null hypotheses will we reject?
 - (b) Now suppose that we wish to control the FWER at level $\alpha = 0.05$. Which null hypotheses will we reject? Justify your answer.
 - (c) Now suppose that we wish to control the FDR at level $q = 0.05$. Which null hypotheses will we reject? Justify your answer.
 - (d) Now suppose that we wish to control the FDR at level $q = 0.2$. Which null hypotheses will we reject? Justify your answer.
 - (e) Of the null hypotheses rejected at FDR level $q = 0.2$, approximately how many are false positives? Justify your answer.
5. For this problem, you will make up p -values that lead to a certain number of rejections using the Bonferroni and Holm procedures.
 - (a) Give an example of five p -values (i.e. five numbers between 0 and 1 which, for the purpose of this problem, we will interpret as p -values) for which both Bonferroni's method and Holm's method

reject exactly one null hypothesis when controlling the FWER at level 0.1.

- (b) Now give an example of five p -values for which Bonferroni rejects one null hypothesis and Holm rejects more than one null hypothesis at level 0.1.
6. For each of the three panels in Figure 13.3, answer the following questions:
- (a) How many false positives, false negatives, true positives, true negatives, Type I errors, and Type II errors result from applying the Bonferroni procedure to control the FWER at level $\alpha = 0.05$?
 - (b) How many false positives, false negatives, true positives, true negatives, Type I errors, and Type II errors result from applying the Holm procedure to control the FWER at level $\alpha = 0.05$?
 - (c) What is the false discovery proportion associated with using the Bonferroni procedure to control the FWER at level $\alpha = 0.05$?
 - (d) What is the false discovery proportion associated with using the Holm procedure to control the FWER at level $\alpha = 0.05$?
 - (e) How would the answers to (a) and (c) change if we instead used the Bonferroni procedure to control the FWER at level $\alpha = 0.001$?

Applied

7. This problem makes use of the `Carseats` dataset in the `ISLP` package.
- (a) For each quantitative variable in the dataset besides `Sales`, fit a linear model to predict `Sales` using that quantitative variable. Report the p -values associated with the coefficients for the variables. That is, for each model of the form $Y = \beta_0 + \beta_1 X + \epsilon$, report the p -value associated with the coefficient β_1 . Here, Y represents `Sales` and X represents one of the other quantitative variables.
 - (b) Suppose we control the Type I error at level $\alpha = 0.05$ for the p -values obtained in (a). Which null hypotheses do we reject?
 - (c) Now suppose we control the FWER at level 0.05 for the p -values. Which null hypotheses do we reject?
 - (d) Finally, suppose we control the FDR at level 0.2 for the p -values. Which null hypotheses do we reject?
8. In this problem, we will simulate data from $m = 100$ fund managers.

```
rng = np.random.default_rng(1)
n, m = 20, 100
X = rng.normal(size=(n, m))
```

These data represent each fund manager's percentage returns for each of $n = 20$ months. We wish to test the null hypothesis that each fund manager's percentage returns have population mean equal to zero. Notice that we simulated the data in such a way that each fund manager's percentage returns do have population mean zero; in other words, all m null hypotheses are true.

- (a) Conduct a one-sample t -test for each fund manager, and plot a histogram of the p -values obtained.
- (b) If we control Type I error for each null hypothesis at level $\alpha = 0.05$, then how many null hypotheses do we reject?
- (c) If we control the FWER at level 0.05, then how many null hypotheses do we reject?
- (d) If we control the FDR at level 0.05, then how many null hypotheses do we reject?
- (e) Now suppose we "cherry-pick" the 10 fund managers who perform the best in our data. If we control the FWER for just these 10 fund managers at level 0.05, then how many null hypotheses do we reject? If we control the FDR for just these 10 fund managers at level 0.05, then how many null hypotheses do we reject?

- (f) Explain why the analysis in (e) is misleading.

Hint: The standard approaches for controlling the FWER and FDR assume that all tested null hypotheses are adjusted for multiplicity, and that no "cherry-picking" of the smallest p -values has occurred. What goes wrong if we cherry-pick?

Index

- accuracy, 415
- activation, 400
- activation function, 401
- additive, 11, 94–98, 110–111
- additivity, 305, 306
- adjusted R^2 , 87, 231, 232, 236–238
- Advertising** data set, 15, 16, 19, 69, 71–73, 77, 78, 80, 82, 83, 85, 87–90, 95, 96, 109–111
- agglomerative clustering, 525
- Akaike information criterion, 87, 231, 232, 236–238
- alternative hypothesis, 76, 559
- analysis of variance, 312
- ANOVA, 587
- area under the curve, 155, 486–487
- argument, 40
- array, 42
- attribute, 42
- AUC, 155
- Auto** data set, 12, 66, 98–101, 129, 197, 202–207, 327, 398
- auto-correlation, 421
- autoregression, 423
- axes, 48
- backfitting, 307, 328
- backpropagation, 429
- backward stepwise selection, 87, 234–235
- bag-of- n -grams, 415
- bag-of-words, 414
- bagging, 11, 24, 331, 343–346, 354, 360–361
- BART, 343, 350, 353, 354, 362–363
- baseline, 93, 145, 161
- basis function, 293–294, 296
- Bayes
 - classifier, 35–37, 147
 - decision boundary, 148
 - error, 35–37
- Bayes’ theorem, 146, 250
- Bayesian, 250–251, 353
- Bayesian additive regression trees, 331, 343, 350, 350, 353, 354, 362–363
- Bayesian information criterion, 87, 231, 232, 236–238
- Benjamini–Hochberg procedure, 575–577
- Bernoulli distribution, 172
- best subset selection, 231, 246
- bias, 31–34, 74, 90, 159, 405
- bias-variance decomposition, 32

- trade-off, 31–34, 38, 111–112, 157, 159, 163, 164, 242, 254, 263, 266, 301, 336, 376, 385
- bidirectional, 425
- Bikeshare** data set, 12, 167–172
- binary, 27, 138
- biplot, 507, 508
- Bonferroni method, 575–577, 585
- Boolean, 53, 176
- boosting, 11, 24, 331, 343, 347–350, 354, 361–362
- bootstrap, 11, 201, 212–214, 343
- Boston** data set, 12, 67, 117, 122, 133, 199, 227, 287, 327, 364, 556
- bottom-up clustering, 525
- boxplot, 62
- BrainCancer** data set, 12, 472–474, 476, 482
- branch, 333
- burn-in, 352
- C-index, 487
- Caravan** data set, 12, 184, 366
- Carseats** data set, 12, 126, 130, 364
- categorical, 2, 27
- censored data, 469–502
- censoring
 - independent, 471
 - interval, 471
 - left, 471
 - mechanism, 471
 - non-informative, 471
 - right, 471
 - time, 470
- chain rule, 429
- channel, 407
- CIFAR100** data set, 406, 409–411, 448, 449
- classification, 2, 11, 27, 34–39, 135–199, 367–382
 - error rate, 338
 - tree, 337–341, 355–358
- classifier, 135
- cluster analysis, 25–26
- clustering, 4, 25–26, 520–535
 - agglomerative, 525
 - bottom-up, 525
 - hierarchical, 521, 525–535
 - K -means, 11, 521–524
- Cochran–Mantel–Haenszel test, 475
- coefficient, 71
- College** data set, 12, 65, 286, 328
- collinearity, 106–110
- concatenation, 41
- conditional probability, 35
- confidence interval, 75–76, 90, 110, 292
- confounding, 144
- confusion matrix, 153, 176
- continuous, 2
- contour, 246
- contour plot, 50
- contrast, 94
- convenience function, 53
- convolution filter, 407
- convolution layer, 407
- convolutional neural network, 406–413
- correlation, 79, 82–83, 530
- count data, 167, 170
- Cox’s proportional hazards model, 480, 483–486
- C_p , 87, 231, 232, 236–238
- Credit** data set, 12, 91, 92, 94, 97, 98, 106–109
- cross-entropy, 405
- cross-validation, 11, 31, 34, 201–211, 231, 252, 270
 - k -fold, 206–209
 - leave-one-out, 204–206
- curse of dimensionality, 115, 193, 266
- data augmentation, 411
- data frame, 55
- Data sets
 - Advertising**, 15, 16, 19, 69, 71–73, 77, 78, 80, 82, 83, 85, 87–90, 95, 96, 109–111
 - Auto**, 12, 66, 98–101, 129, 197, 202–207, 327, 398
 - Bikeshare**, 12, 167–172

- Boston**, 12, 67, 117, 122, 133, 199, 227, 287, 327, 364, 556
- BrainCancer**, 12, 472–474, 476, 482
- Caravan**, 12, 184, 366
- Carseats**, 12, 126, 130, 364
- CIFAR100**, 406, 409–411, 448, 449
- College**, 12, 65, 286, 328
- Credit**, 12, 91, 92, 94, 97, 98, 106–109
- Default**, 12, 136–139, 141–144, 152–156, 160, 161, 225, 226, 466
- Fund**, 12, 567–570, 572, 575, 576, 585, 588, 589
- Heart**, 339, 340, 344–347, 352, 353, 382, 383
- Hitters**, 12, 332, 333, 336, 338, 339, 366, 425, 426, 437, 446
- IMDb**, 413, 415, 416, 418, 420, 437, 458, 467
- Income**, 16–18, 21–23
- Khan**, 12, 579–581, 583, 590, 593
- MNIST**, 402–404, 406, 430, 431, 441, 444, 445, 448
- NCI60**, 4, 5, 12, 546, 548–550
- NYSE**, 12, 422–424, 466, 467
- OJ**, 12, 365, 398
- Portfolio**, 12
- Publication**, 12, 482–487
- Smarket**, 2, 3, 12, 173, 184, 196
- USArrests**, 12, 507, 508, 510, 512, 513, 515, 516, 518, 519
- Wage**, 1, 2, 8, 9, 12, 290, 291, 293, 295, 297–300, 302–306, 309, 315, 327
- Weekly**, 12, 196, 226
- data type, 42
- decision function, 387
- decision tree, 11, 331–342
- deep learning, 399
- Default** data set, 12, 136–139, 141–144, 152–156, 160, 161, 225, 226, 466
- degrees of freedom, 30, 266, 295, 296, 301
- dendrogram, 521, 525–530
- density function, 146
- dependent variable, 15
- derivative, 296, 300
- detector layer, 410
- deviance, 232
- dictionary, 66
- dimension reduction, 230, 253–262
- discriminant function, 149
- discriminant method, 146–161
- dissimilarity, 530–532
- distance
 - correlation-based, 530–532, 554
 - Euclidean, 509, 522, 523, 529–532
- double descent, 431–435
- double-exponential distribution, 251
- dropout, 406, 431
- dummy variable, 91–94, 138, 142, 292
- early stopping, 430
- effective degrees of freedom, 301
- eigen decomposition, 506, 516
- elbow, 548
- embedding, 418
- embedding layer, 419
- ensemble, 343–354
- entropy, 337–339, 363
- epochs, 430
- error
 - irreducible, 17, 30
 - rate, 34
 - reducible, 17
 - term, 16
- Euclidean distance, 509, 522, 523, 529–532, 554
- event time, 470
- exception, 45
- expected value, 18
- exploratory data analysis, 504
- exponential, 173
- exponential family, 173
- F-statistic, 84

- factor, 92
- factorial, 170
- failure time, 470
- false
 - discovery proportion, 155, 573
 - discovery rate, 558, 573–577, 579–582
 - negative, 155, 562
 - positive, 155, 562, 563
 - positive rate, 155, 156, 382
- family-wise error rate, 565–573, 577
- feature, 15
- feature map, 406
- feature selection, 230
- featurize, 414
- feed-forward neural network, 400
- figure, 48
- fit, 21
- fitted value, 101
- flattening, 424
- flexible, 21
- floating point, 43
- forward stepwise selection, 86, 87, 233–234, 268
- function, 40
- Fund** data set, 12, 567–570, 572, 575, 576, 585, 588, 589
- Gamma, 173
- Gaussian (normal) distribution, 146, 147, 150, 172, 561
- generalized additive model, 5, 24, 162, 289, 290, 305–309, 319
- generalized linear model, 5, 135, 167–174, 217
- generative model, 146–161
- Gini index, 337–339, 345, 346, 363
- global minimum, 427
- gradient, 428
- gradient descent, 427
- Harrell’s concordance index, 487
- hazard function, 476–478
 - baseline, 478
- hazard rate, 476
- Heart** data set, 339, 340, 344–347, 352, 353, 382, 383
- heatmap, 50
- helper, 311
- heteroscedasticity, 103, 168
- hidden layer, 400
- hidden units, 400
- hierarchical clustering, 525–530
 - dendrogram, 525–528
 - inversion, 529
 - linkage, 529–530
- hierarchical principle, 96
- high-dimensional, 86, 234, 263
- hinge loss, 385
- Hitters** data set, 12, 332, 333, 336, 338, 339, 366, 425, 426, 437, 446
- hold-out set, 202
- Holm’s method, 568, 576, 585
- hypergeometric distribution, 501
- hyperparameter, 187
- hyperplane, 367–372
- hypothesis test, 76–77, 84, 103, 558–583
- IMDb** data set, 413, 415, 416, 418, 420, 437, 458, 467
- imputation, 515
- Income** data set, 16–18, 21–23
- increment, 60
- independent variable, 15
- indexable, 186
- indicator function, 292
- inference, 17, 18
- inner product, 379, 380
- input layer, 400
- input variable, 15
- integral, 301
- interaction, 70, 89, 95–98, 110–111, 308
- intercept, 71, 72
- interpolate, 432
- interpretability, 229
- inversion, 529
- irreducible error, 17, 36, 90, 110
- iterator, 312
- joint distribution, 158
- K-means clustering, 11, 521–524
- K-nearest neighbors, 135, 164–167
 - classifier, 11, 36–37
 - regression, 111–115

- Kaplan–Meier survival curve, 472–474, 483
- kernel, 379–382, 384, 394
 - linear, 380
 - non-linear, 377–382
 - polynomial, 380, 382
 - radial, 381–383, 390
- kernel density estimator, 159
- keyword, 46
- Khan** data set, 12, 579–581, 583, 590, 593
- knot, 290, 294, 296–299
- ℓ_1 norm, 244
- ℓ_2 norm, 242
- lag, 422
- Laplace distribution, 251
- lasso, 11, 24, 244–251, 265–266, 336, 385, 484
- leaf, 333, 526
- learning rate, 429
- least squares, 5, 21, 71–72, 140, 141, 229
 - line, 73
 - weighted, 103
- level, 92
- leverage, 104–106
- likelihood function, 141
- linear, 2, 69–115
- linear combination, 128, 230, 253, 505
- linear discriminant analysis, 5, 11, 135, 138, 147–155, 164–167, 377, 382
- linear kernel, 380
- linear model, 20, 69–115
- linear regression, 5, 11, 69–115, 172–173
 - multiple, 80–90
 - simple, 70–80
- link function, 172, 173
- linkage, 529–530, 548
 - average, 529–530
 - centroid, 529–530
 - complete, 526, 529–530
 - single, 529–530
- list, 41
- list comprehension, 123
- local minimum, 427
- local regression, 290
- log odds, 145
- log-rank test, 474–476, 483
- logistic function, 139
- logistic regression, 5, 11, 25, 135, 138–144, 164–167, 172–173, 308–309, 377, 384–385
 - multinomial, 145, 163
 - multiple, 142–144
- logit, 140
- loss function, 300, 385
- low-dimensional, 262
- LSTM RNN, 420
- main effects, 96
- majority vote, 344
- Mallow’s C_p , 87, 231, 232, 236–238
- Mantel–Haenszel test, 475
- margin, 370, 385
- marginal distribution, 158
- Markov chain Monte Carlo, 353
- matrix completion, 515
- matrix multiplication, 10
- maximal margin
 - classifier, 367–372
 - hyperplane, 370
- maximum likelihood, 139–141, 143, 170
- mean squared error, 28
- mesh, 53
- method, 43
- minibatch, 429
- misclassification error, 35
- missing at random, 515
- missing data, 56, 515–520
- mixed selection, 87
- MNIST** data set, 402–404, 406, 430, 431, 441, 444, 445, 448
- model assessment, 201
- model selection, 201
- module, 42
- multicollinearity, 108, 266
- multinomial logistic regression, 145, 163
- multiple testing, 557–583
- multi-task learning, 403
- multivariate Gaussian, 150

- multivariate normal, 150
- naive Bayes, 135, 158–161, 164–167
- namespace, 116
- natural spline, 297, 298, 301, 317
- NCI60 data set, 4, 5, 12, 546, 548–550
- negative binomial, 173
- negative predictive value, 155, 156
- neural network, 5, 399
- node
 - internal, 333
 - purity, 337–339
 - terminal, 333
- noise, 21, 252
- non-linear, 2, 11, 289–329
 - decision boundary, 377–382
 - kernel, 377–382
- non-parametric, 20, 22–23, 111–115, 193
- normal (Gaussian) distribution, 146, 147, 150, 172, 476, 561
- notebook, 40
- null, 152
 - distribution, 561, 578
 - hypothesis, 76, 559
 - model, 87, 231, 245
- null rate, 186
- NYSE data set, 12, 422–424, 466, 467
- Occam’s razor, 426
- odds, 140, 145, 195
- OJ data set, 12, 365, 398
- one-hot encoding, 92, 126, 403
- one-standard-error rule, 240
- one-versus-all, 384
- one-versus-one, 384
- one-versus-rest, 384
- optimal separating hyperplane, 370
- optimism of training error, 30
- ordered categorical variable, 315
- orthogonal, 257, 506
 - basis, 125
- out-of-bag, 345
- outlier, 103–104
- output variable, 15
- over-parametrized, 465
- overdispersion, 172
- overfitting, 21, 23, 25, 30–31, 88, 152, 233, 371
- p-value, 77, 82, 560–562, 578–579
 - adjusted, 586
- package, 42
- parameter, 71
- parametric, 20–22, 111–115
- partial least squares, 254, 260–262, 282
- partial likelihood, 480
- path algorithm, 249
- permutation, 578
- permutation approach, 577–582
- perpendicular, 257
- Poisson distribution, 169, 172
- Poisson regression, 135, 167–173
- polynomial
 - kernel, 380, 382
 - regression, 98–99, 289–292, 294–295
- pooling, 410
- population regression line, 73
- Portfolio data set, 12
- positive predictive value, 155, 156
- posterior
 - distribution, 251
 - mode, 251
 - probability, 147
- power, 108, 155, 563
- precision, 155
- prediction, 17
 - interval, 90, 110
- predictor, 15
- principal components, 505
 - analysis, 11, 254–260, 504–515
 - loading vector, 505, 506
 - missing values, 515–520
 - proportion of variance explained, 510–515, 547
 - regression, 11, 254–260, 280–282, 504, 515
 - score vector, 506
 - scree plot, 514–515
- prior
 - distribution, 251
 - probability, 146
- probability density function, 477, 478

- projection, 230
- proportional hazards assumption, 478
- pruning, 336
 - cost complexity, 336
 - weakest link, 336
- Publication data set, 12, 482–487
- Python objects and functions
 - %capture, 458
 - iloc[], 58
 - loc[], 57
 - AgglomerativeClustering(), 543
 - anova(), 313
 - anova_lm(), 125, 129, 312, 313
 - axhline(), 122, 551
 - axline(), 121, 129, 329
 - BART(), 362
 - biplot, 537
 - boot_SE(), 223
 - boxplot(), 62, 66
 - bs(), 315, 327
 - BSpline(), 315
 - clone(), 222
 - columns.drop(), 122
 - compute_linkage(), 544
 - confusion_table(), 176
 - contour(), 50
 - corr(), 129, 174
 - cost_complexity_pruning_path(), 357
 - CoxPHFitter(), 491
 - cross_val_predict(), 270
 - cross_validate(), 218, 219, 226
 - cumsum(), 539
 - cut_tree(), 545
 - data.frame(), 227
 - Dataset, 440
 - decision_function(), 392
 - DecisionTreeClassifier(), 354, 355
 - DecisionTreeRegressor(), 354
 - def, 121
 - dendrogram(), 544
 - describe(), 62, 66
 - dir(), 116
 - drop(), 179
 - dropna(), 56, 268, 461
 - DTC(), *see* DecisionTreeClassifier()
 - DTR(), *see* DecisionTreeRegressor()
 - dtype, 43
 - ElasticNetCV(), 279
 - enumerate(), 217
 - export_text(), 356
 - export_tree(), 365
 - fit(), 118, 181, 218
 - fit_transform(), 119
 - for, 59
 - GaussianNB(), 182
 - GBR(), *see* GradientBoostingRegressor()
 - get_dummies(), 461
 - get_influence(), 121
 - get_prediction(), 120, 314
 - get_rdataset(), 535
 - glm(), 313
 - glob(), 437
 - GradientBoostingClassifier(), 361
 - GradientBoostingRegressor(), 354, 361
 - GridSearchCV(), 276
 - groupby(), 490
 - hist(), 62
 - iloc[], 58, 59
 - import, 42
 - imshow(), 50, 449
 - ISLP.bart, 362
 - ISLP.cluster, 544
 - json, 437
 - KaplanMeierFitter(), 502
 - keras, 437
 - KFold(), 219
 - KMeans(), 542, 543
 - Kmeans(), 542
 - KNeighborsClassifier(), 183
 - lambda, 58
 - LDA(), *see* LinearDiscriminantAnalysis()
 - legend(), 132
 - lifelines, 490
 - LinearDiscriminantAnalysis(), 174, 179
 - LinearGAM(), 317
 - LinearRegression(), 280

- `load_data()`, 117
- `loc[]`, 58, 59, 177
- `log_loss()`, 355
- `LogisticGAM()`, 323
- `logrank_test()`, 490
- `lowess()`, 324
- `matplotlib`, 48
- `max()`, 66
- `mean()`, 48
- `median()`, 197
- `min()`, 66
- `MNIST()`, 444
- `ModelSpec()`, 116–118, 122, 124, 267
- `MS()`, *see* `ModelSpec()`
- `mult_test()`, *see* `multipletests()`
- `multipletests()`, 586
- `multipletests()`, 583, 589
- `multivariate_logrank_test()`, 496
- `NaturalSpline()`, 317, 319
- `ndim`, 42
- `nn.RNN()`, 461
- `normal()`, 132, 286, 555
- `np`, *see* `numpy`
- `np.all()`, 54, 180
- `np.allclose()`, 190
- `np.any()`, 54
- `np.arange()`, 51
- `np.argmax()`, 122
- `np.array()`, 42
- `np.concatenate()`, 133
- `np.corrcoef()`, 46, 554
- `np.empty()`, 224
- `np.isnan()`, 268
- `np.ix_()`, 53
- `np.linalg.svd()`, 539
- `np.linspace()`, 50
- `np.logspace()`, 318
- `np.mean()`, 47, 176
- `np.nan`, 60
- `np.nanmean()`, 541
- `np.percentile()`, 228
- `np.power()`, 219
- `np.random.choice()`, 553
- `np.random.default_rng()`, 46, 47
- `np.random.normal()`, 45
- `np.sqrt()`, 45
- `np.squeeze()`, 457
- `np.std()`, 47
- `np.sum()`, 43
- `np.var()`, 47
- `np.where()`, 180
- `ns()`, 317
- `numpy`, 42, 555
- `os.chdir()`, 55
- `outer()`, 219
- `pairwise_distances()`, 554
- `pairwise_tukeyhsd()`, 587
- `pandas`, 55
- `params`, 175
- `partial()`, 222, 269
- `PCA()`, 280, 537, 540, 554
- `pd`, *see* `pandas`
- `pd.crosstab()`, 555
- `pd.cut()`, 315
- `pd.get_dummies()`, 314
- `pd.plotting.scatter_matrix()`, 62
- `pd.qcut()`, 314, 315
- `pd.read_csv()`, 55, 556
- `pd.Series()`, 62
- `Pipeline()`, 275
- `plot()`, 48, 61, 356, 490
- `plot.scatter()`, 120
- `plot_gam()`, 321
- `plot_svm()`, 398
- `PLSRegression()`, 282
- `poly()`, 125, 313, 327
- `predict()`, 175, 178, 181, 216, 218, 323, 358
- `predict_survival_function()`, 493
- `print()`, 40
- `pvalues`, 175
- `pygam`, 307, 317
- `pytorch_lightning`, 435
- `QDA()`, *see* `QuadraticDiscriminantAnalysis()`
- `QuadraticDiscriminantAnalysis()`, 174, 181
- `random()`, 555
- `RandomForestRegressor()`, 354, 360
- `read_image()`, 436
- `reindex()`, 461
- `reshape()`, 43

- `return`, 198
- `RF()`, *see* `RandomForestRegressor()`
- `rng`, *see* `np.random.default_rng()`
- `rng.choice()`, 60
- `rng.standard_normal()`, 60
- `roc_curve()`, 392
- `RocCurveDisplay.from_estimator()`, 387
- `savefig()`, 50
- `scatter()`, 49, 61
- `scipy.interpolate`, 315
- `score()`, 218, 461
- `seed_everything()`, 436
- `set_index()`, 57
- `set_title()`, 49
- `set_xlabel()`, 49
- `set_xscale()`, 198
- `set_ylabel()`, 49
- `set_yscale()`, 198
- `shape`, 43
- `ShuffleSplit()`, 219
- `sim_time()`, 495
- `SimpleDataModule()`, 441
- `SimpleModule.classification()`, 446
- `SimpleModule.regression()`, 442
- `skl`, *see* `sklearn.linear_model`
- `skl.ElasticNet()`, 273, 277
- `skl.ElasticNet.path`, 274
- `skl.ElasticNet.path()`, 273
- `sklearn`, 118, 181
- `sklearn.ensemble`, 360
- `sklearn.linear_model`, 267
- `sklearn.model_selection`, 267
- `sklearn_selected()`, 269
- `sklearn_selection_path()`, 270
- `sklearn_sm()`, 218
- `skm`, *see* `sklearn.model_selection`
- `skm.cross_val_predict()`, 271
- `skm.KFold()`, 271
- `skm.ShuffleSplit()`, 272
- `slice()`, 51, 462
- `sm`, *see* `statsmodels`
- `sm.GLM()`, 174, 192, 226
- `sm.Logit()`, 174
- `sm.OLS()`, 118, 129, 174, 319
- `StandardScaler()`, 185, 438, 537, 555
- `statsmodels`, 116, 173
- `std()`, 186
- `Stepwise()`, 269
- `str.contains()`, 59
- `subplots()`, 48
- `sum()`, 43, 268
- `summarize()`, 118, 129, 223, 226
- `summary()`, 119, 322, 587
- `super()`, 440
- `SupportVectorClassifier()`, 387, 389–391, 393
- `SupportVectorRegression()`, 394
- `SVC()`, *see* `SupportVectorClassifier()`
- `svd()`, 539
- `SVR()`, *see* `SupportVectorRegression()`
- `TensorDataset()`, 441
- `to_numpy()`, 437
- `torch`, 435
- `torchinfo`, 436
- `torchmetrics`, 436
- `torchvision`, 436
- `ToTensor()`, 444
- `train_test_split()`, 186, 216
- `transform()`, 118, 119
- `ttest_1samp()`, 584
- `ttest_ind()`, 590
- `ttest_rel()`, 587
- `tuple`, 43
- `uniform()`, 555
- `value_counts()`, 66
- `var()`, 536
- `variance_inflation_factor()`, 116, 124
- `VIF()`, *see* `variance_inflation_factor()`
- `where()`, 355
- `zip()`, 60, 312
- `q-values`, 589
- `quadratic`, 98
- `quadratic discriminant analysis`, 4, 135, 156–157, 164–167

- qualitative, 2, 27, 91, 135, 167, 202
 - variable, 91–94
- quantitative, 2, 27, 91, 135, 167, 202
- radial kernel, 381, 383, 390
- random forest, 11, 331, 343, 346–347, 354, 360–361
- random seed, 46
- re-sampling, 577–582
- recall, 155
- receiver operating characteristic (ROC), 154, 382–383
- recommender systems, 516
- rectified linear unit, 401
- recurrent neural network, 416–427
- recursive binary splitting, 334, 337, 338
- reducible error, 17, 90
- regression, 2, 11, 27
 - local, 289, 290, 304–305
 - piecewise polynomial, 294–295
 - polynomial, 289–292, 299
 - spline, 289, 294
 - tree, 331–337, 358–360
- regularization, 230, 240, 406, 484–486
- ReLU, 401
- resampling, 201–214
- residual, 71, 81
 - plot, 100
 - standard error, 75, 77–78, 88–89, 109
 - studentized, 104
 - sum of squares, 71, 79, 81
- residuals, 263, 348
- response, 15
- ridge regression, 11, 240–244, 385, 484
- risk set, 473
- robust, 374, 376, 535
- ROC curve, 154, 382–383, 486–487
- R^2 , 77–80, 88, 109, 238
- rug plot, 314
- scale equivariant, 242
- Scheffé’s method, 572
- scree plot, 512, 514–515
- elbow, 514
- semi-supervised learning, 27
- sensitivity, 153, 155, 156
- separating hyperplane, 367–372
- Seq2Seq, 425
- sequence, 41
- shrinkage, 230, 240, 484–486
 - penalty, 240
- sigmoid, 401
- signal, 252
- signature, 45
- singular value decomposition, 539
- slack variable, 375
- slice, 51
- slope, 71, 72
- Smarket** data set, 2, 3, 12, 173, 184, 196
- smoother, 308
- smoothing spline, 290, 300–303
- soft margin classifier, 372–374
- soft-thresholding, 250
- softmax, 145, 405
- sparse, 244, 252
- sparse matrix format, 414
- sparsity, 244
- specificity, 153, 155, 156
- spline, 289, 294–303
 - cubic, 296
 - linear, 296
 - natural, 297, 301
 - regression, 289, 294–299
 - smoothing, 30, 290, 300–303
 - thin-plate, 22
- standard error, 75, 101
- standardize, 185
- statistical model, 1
- step function, 111, 289, 292–293
- stepwise model selection, 11, 231, 233
- stochastic gradient descent, 429
- string, 41
- string interpolation, 490
- stump, 349
- subset selection, 230–240
- subtree, 336
- supervised learning, 25–27, 261
- support vector, 371, 376, 385
 - classifier, 367, 372–377
 - machine, 5, 11, 24, 377–386

- regression, 386
- survival
 - analysis, 469–502
 - curve, 472, 483
 - function, 472
 - time, 470
- synergy, 70, 89, 95–98, 110–111
- systematic, 16
- t-distribution, 77, 165
- t-statistic, 76
- t-test
 - one-sample, 583, 584, 588
 - paired, 587
 - two-sample, 559, 570, 571, 577–581, 584, 590
- test
 - error, 35, 37, 176
 - MSE, 28–32
 - observations, 28
 - set, 30
 - statistic, 559
- theoretical null distribution, 577
- time series, 101
- total sum of squares, 79
- tracking, 102
- train, 21
- training
 - data, 20
 - error, 35, 37, 176
 - MSE, 28–31
- transformer, 311
- tree, 331–342
- tree-based method, 331
- true negative, 155
- true positive, 155
- true positive rate, 155, 156, 382
- truncated power basis, 296
- Tukey’s method, 571, 585, 587
- tuning parameter, 187, 240, 484
- two-sample *t*-test, 474
- Type I error, 155, 562–565
- Type I error rate, 563
- Type II error, 155, 563, 568, 584
- unsupervised learning, 25–27, 255, 260, 503–552
- USArrests data set, 12, 507, 508, 510, 512, 513, 515, 516, 518, 519
- validation set, 202
 - approach, 202–204
- variable, 15
 - dependent, 15
 - dummy, 91–94, 97–98
 - importance, 346, 360
 - independent, 15
 - indicator, 35
 - input, 15
 - output, 15
 - qualitative, 91–94, 97–98
 - selection, 86, 230, 244
- variance, 18, 31–34, 159
 - inflation factor, 108–110, 123
- varying coefficient model, 305
- Wage data set, 1, 2, 8, 9, 12, 290, 291, 293, 295, 297–300, 302–306, 309, 315, 327
- weak learner, 343
- weakest link pruning, 336
- Weekly data set, 12, 196, 226
- weight freezing, 412, 419
- weight sharing, 418
- weighted least squares, 103, 304
- weights, 404
- with replacement, 214
- within class covariance, 150
- wrapper, 217